LLM-BASED WHEEL HUMAN INTERACTION ROBOT FOR RECEPTIONIST AND HUMAN ASSISTANT

KIRAWUT CHALERMKITPAISAN
KRITTIN SAKHARIN
NATCHA RUNGRUANG
NATWASA MANOMAIWIBOON

A THESIS SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENT FOR THE DEGREE OF
BACHELOR OF ENGINEERING IN ROBOTICS AND AI ENGINEERING
SCHOOL OF ENGINEERING
KING MONGKUT'S INSTITUTE OF TECHNOLOGY LADKRABANG
2025

KMITL-2025-EN-X-XXX-XXX

LLM-BASED WHEEL HUMAN INTERACTION ROBOT FOR RECEPTIONIST AND HUMAN ASSISTANT

KIRAWUT CHALERMKITPAISAN
KRITTIN SAKHARIN
NATCHA RUNGRUANG
NATWASA MANOMAIWIBOON

A THESIS SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENT FOR THE DEGREE OF
BACHELOR OF ENGINEERING IN ROBOTICS AND AI ENGINEERING
SCHOOL OF ENGINEERING
KING MONGKUT'S INSTITUTE OF TECHNOLOGY LADKRABANG
2025

KMITL-2025-EN-X-XXX-XXX

COPYRIGHT 2025
SCHOOL OF ENGINEERING
KING MONGKUT'S INSTITUTE OF TECHNOLOGY LADKRABANG

Thesis	LLM-based Wheel Human Interaction Robot for Receptionist and Human Assistant
Student	Mr. Kirawut Chalermkitpaisan, Mr. Krittin Sakharin, Ms. Natcha Rungruang, Ms Natwasa Manomaiwiboon
Student ID.	65011333, 65011356, 65011386, 65011402
Degree	Bachelor of Engineering
Program	Robotics and AI Engineering
Year	2025
Thesis Advisor	Asst. Prof. Dr. Sarucha Yanyong

ABSTRACT IN ENGLISH

A good abstract writing should focus only on the key and interesting aspects of the research. It should emphasize the specific highlights of the research, with the research work being clear, concise, and meeting certain criteria for a good abstract. For example, the word count should be between 200-250 words, or approximately no more than half of an A4 page. Before writing the abstract, one should read and understand their own complete research work to identify various interesting points that can engage general readers, making them want to read the full research. There should be no interpretation or criticism using personal opinions. Complex or aggressive language should be avoided. Local jargon, unnecessary abbreviations, or symbols should not be used as they may lead to misunderstanding. There should be no numerical references, diagrams, tables, various statistical formulas, or equations in the abstract, unless necessary to display analytical results. The abstract may have multiple paragraphs to enhance reader comprehension in each section. Avoid referencing the research work of others in the abstract. The structure of the abstract should also be carefully considered, including font type, font size, page margin, printing standards, and an acceptable format. Proofreading the writing by reading it multiple times will improve the quality of the abstract, making it acceptable for research presentations.

ACKNOWLEDGEMENT

Acknowledgement refers to an announcement of appreciation for virtue. It specifically acknowledges the goodness of individuals who provided assistance leading to the successful completion of research, a thesis, or an academic report. During the process of conducting research, a thesis, or academic work, individuals may receive assistance, guidance, or support that contributes to the successful completion of the task. In recognition of this aid and in gratitude, individuals express their appreciation by writing a Certificate of Appreciation, which is included at the beginning of the research, thesis, or academic work. This serves to demonstrate satisfaction and heartfelt gratitude for the goodness of those who assisted. In essence, a Certificate of Appreciation is a declaration of gratitude.

Name Surname

TABLE OF CONTENTS

Page

Abstract ii

ACKNOWLEDGEMENT iii

TABLE OF CONTENTS ix

LIST OF TABLES x

LIST OF FIGURES xi

LIST OF ABBREVIATIONS AND SYMBOLS xii

CHAPTER 1 INTRODUCTION 1

1.1 STATEMENT AND SIGNIFICANCE OF THE PROBLEM 1

1.2 GOAL AND OBJECTIVE OF THE STUDY 1

1.3 HYPOTHESIS 2

1.3.1 Autonomous Navigation and Obstacle Avoidance 2

1.3.2 Effectiveness of Natural Language Interaction 2

1.3.3 Improved Response Accuracy through Retrieval-Augmented Generation (RAG) 2

1.3.4 Efficiency of Data Management System 2

1.3.5 Usability of Voice-Based Interaction 2

1.3.6 User Interface Accessibility and Acceptance 2

1.3.7 Real-Time System Performance 2

1.4 SCOPE OF THE STUDY 3

1.5 PROCESS OF THE STUDY 3

1.5.1 Robot Mechanical Design 3

1.5.2 Motor Selection and Placement 3

1.5.3 Control System Design 3

1.5.4 Navigation and Interaction 3

1.5.5 Testing and Evaluation 3

1.6 ASSUMPTIONS 4

1.6.1 Hardware Capability 4

1.6.2 Power Availability 4
1.6.3 Environmental Conditions 4
1.6.4 Network Connectivity 4
1.7 LIMITATION OF THE STUDY 4
1.7.1 Technical Constraints 4
1.7.1.1 Non-Technical Design Constraints 5
1.8 DEFINITIONS 6

CHAPTER 2 LITERATURE REVIEW 7
2.1 MECHANICAL DESIGN 7
2.1.1 Torque 7
2.1.2 Kinematics 8
2.2 ELECTRICAL DESIGN 9
2.2.1 Power Supply Architecture and Electrical Safety 9
2.2.1.1 Series and Parallel Connection 9
2.2.1.2 Electrical Safety Considerations 9
2.2.2 DC to DC Buck Converter 10
2.2.3 Power Consumption Analysis 10
2.2.4 Electrical Interconnection and Power Distribution 12
2.3 SOFTWARE DESIGN 13
2.3.1 Mobile Robot 13
2.3.1.1 ROS 2 (Robot Operating System 2) 13
2.3.1.2 ROS Navigation Stack 13
2.3.1.3 AMCL (Adaptive Monte Carlo Localization) 13
2.3.2 Database Management 14
2.3.2.1 Retrieval-Augmented Generation (RAG) 14
2.3.2.2 Vector Database 14
2.3.2.3 Embeddings 14
2.3.2.4 Milvus 14
2.3.2.5 SQL (Structured Query Language) 15
2.3.3 Artificial Intelligence 15
2.3.3.1 Face Recognition 15
2.3.3.2 Natural Language Processing (NLP) 15
2.3.3.3 Large Language Model (LLM) 16
2.3.3.4 Text-to-Speech (TTS) 16
2.3.3.5 Automatic Speech Recognition (ASR) 16
2.3.3.6 Voice Activity Detection (VAD) 17
2.3.3.7 Multi-Object Tracking 17

CHAPTER 3 METHODOLOGY **18**
3.1 System Overview 18
3.1.1 Distributed Node Architecture 18
3.1.2 End-to-End Interaction Pipeline 19
3.2 Electrical System Design 20
3.2.1 Power Supply Architecture 20
3.3 ROS System Architecture and Development 20
3.3.1 ROS Architecture Overview 20
3.3.2 Hardware Communication and Motor Control 21
3.3.3 Omnidirectional Kinematics 21
3.3.4 IMU Processing and Odometry Estimatio 21
3.3.5 IMU Processing and Odometry Estimation 21
3.3.6 LiDAR Integration and Scan Filtering 22
3.3.7 Scan Matching Odometry 22
3.3.8 Localisation and Navigation Stack 22
3.3.8.1 Map-Based Localisation with AMCL 22
3.3.8.2 Auto-Localisation at Initial Position 22
3.3.8.3 Path Planning 22
3.3.8.4 Stuck Detection and Recovery 23
3.3.9 Navigation State Machine 23
3.3.9.1 Idle and Stop State 23
3.3.9.2 Autonomous Roaming State 23
3.3.9.3 Goal-Directed Navigation State 23
3.3.9.4 HTTP Bridge for External System Integration 23
3.4 Software Design and Development 24
3.4.1 Software Architecture Overview 24
3.4.2 Database Design 24
3.4.2.1 Milvus Vector Database 24
3.4.2.2 MySQL Relational Database 25
3.4.2.3 Database Synchronisation 26
3.4.3 LLM Integration Module 26
3.4.3.1 Query Routing Strategy 26
3.4.3.2 Personalised Prompt Generation 27
3.4.3.3 Output Schema and Intent Classification 28
3.4.3.4 Session Memory and Persistence 28
3.4.4 Speech-to-Text and Grammar Correction 28
3.4.4.1 ASR via Typhoon2-Audio 28
3.4.4.2 Grammar Normalisation 29
3.4.4.3 Robustness Mechanisms 29

3.4.5 Text-to-Speech Module 29
3.4.5.1 Text Normalisation 30
3.4.5.2 Asynchronous Dispatch 30
3.4.6 Audio Sidecar Service 30
3.4.6.1 TTS Synthesis Flow 31
3.4.6.2 STT Transcription Flow 31
3.4.7 Inference Pipeline 31
3.4.8 Vision and Face Recognition Subsystem 33
3.4.8.1 Detection and Tracking 34
3.4.8.2 Head Pose Estimation and Frontal Gate 34
3.4.8.3 Face Recognition 34
3.4.8.4 WebSocket Event Emission 34
3.4.9 Edge Audio Coordination Subsystem 35
3.4.9.1 Concurrent Task Architecture 36
3.4.9.2 Voice Activity Detection and Transcription 36
3.4.9.3 Detection-Speech Combiner Logic 36
3.4.9.4 Audio Playback and Watchdog 37
3.4.10 Expressive Robot Interface Subsystem 37
3.4.10.1 Expression System 37
3.4.10.2 IPC Architecture 38
3.4.11 Autonomous Navigation Subsystem 38
3.4.11.1 Hardware Layer and Odometry 39
3.4.11.2 Localisation and Path Planning 39
3.4.11.3 HTTP Bridge and Navigation State Machine 39
3.4.12 Inter-subsystem Communication Architecture 40
3.4.13 System Data Flow and Interaction Scenarios 41
3.4.13.1 Scenario A: Registered Student – Conversational Query 41
3.4.13.2 Scenario B: Navigation Request 41
3.4.14 Session and State Management 42
3.4.14.1 Robot Interaction State 42
3.4.14.2 Student Session Lifecycle 42
3.4.14.3 Navigation State Machine 42

CHAPTER 4 REFERENCE AND CITATION 44
4.1 Development Environment and Dependencies 44
4.2 Database Setup and Knowledge Ingestion 44
4.3 API Endpoint Behaviour 44
4.4 Performance Evaluation 45
4.5 System Evaluation 45

4.6 System Robustness 45
4.7 Deployment and System Startup 46

CHAPTER 5 BIBLIOGRAPHY AND REFERENCES 47
5.1 GUIDELINES FOR WRITING BIBLIOGRAPHY 47
5.2 GUIDELINES FOR WRITING REFERENCES 47
5.3 METHOD OF LISTING BIBLIOGRAPHY 47
5.3.1 Authors 47
5.3.2 Article Titles 48
5.3.3 Book or Journal Titles 48
5.3.4 Editions 48
5.3.5 Place of Publication and Publisher 48
5.3.6 Year of Publication 49
5.3.7 Example of Bibliography Format 49
5.3.7.1 Book 49
5.3.7.2 Translated Books 50
5.3.7.3 Translated Books with Unspecified Original Author in the Original Language 50
5.3.7.4 Books Authored by Organizations 50
5.3.7.5 Editors or Compilers 50
5.3.7.6 Conference Proceeding or Articles from Books 51
5.3.7.7 Journal Articles 51
5.3.7.8 Thesis and Dissertation 51
5.3.7.9 Reviews 51
5.3.7.10 Articles in Encyclopedias or Dictionaries 52
5.3.7.11 Newspaper Articles 52
5.3.7.12 News Monitoring and Facts Checking in Newspapers Over a Period 52
5.3.7.13 Newsletters, Photocopies, and Unpublished Documents 52
5.3.7.14 Government Publications and Other Official Documents 52
5.3.7.15 Interviews 53
5.3.7.16 Audiovisual Materials 53
5.3.7.17 Patents 53
5.3.7.18 Electronic Media 53
5.3.8 Example Formats for References Citation 54
5.3.8.1 Book 54
5.3.8.2 Articles from Journals, Books, or Conference Proceedings 54
5.3.8.3 Thesis and Dissertation 55
5.3.8.4 Patents 55
5.3.8.5 Visual Materials and Other Media 55

5.3.8.6 Electronic Media 55

BIBLIOGRAPHY **56**

Appendix A Guidelines and Procedures For Thesis Writing **57**

AUTHOR BIOGRAPHY **60**

LIST OF TABLES

Table		Page
3.1	System Components and Specifications	18
3.2	Network Endpoints and Communication Protocols	19
3.3	Milvus Vector Database Collections and Retrieval Signals	27
3.4	STT Backend Comparison for Edge Audio Coordination Subsystem	29
3.5	Audio Sidecar API Endpoints (Port 8001)	31
3.6	AI Inference Pipeline Latency Summary (Single GPU Server)	33
3.7	RaspiReceive Subsystem Configuration Parameters	37
3.8	Robot Face Expression States	38
3.9	Navigation State Definitions	40
3.10	Communication Links and Protocol Selection Rationale	41
4.1	Pipeline Timing Benchmarks (approximate, single GPU)	45

LIST OF FIGURES

Figure **Page**

3.1 System Overview: Distributed Node Architecture and Communication Pathways 19

3.2 Overview of Power Supply Architecture 20

3.3 RAG Query Routing: Keyword Classifier Directing Queries to Six Retrieval Channels 27

3.4 Audio Sidecar Service: TTS Synthesis and STT Transcription Processing Flows (Port 8001) 30

3.5 AI Inference Pipeline: Eight Sequential Processing Stages from Input Filter to Asynchronous Dispatch 32

3.6 JetsonVision Pipeline: Face Detection, Multi-Object Tracking, Head Pose Gating, and Recognition . 33

3.7 RaspiReceive Pipeline: Pre-emphasis, Voice Activity Detection, STT, Combiner Gating, and Server Dispatch 35

3.8 Robot Face Expression Flow: State Transitions Driven by Interaction Lifecycle Events..... 38

3.9 Autonomous Navigation State Machine: Goal Dispatch, Stuck Detection (360° Recovery), and Retry Logic 40

LIST OF ABBREVIATIONS AND SYMBOLS

C	Specific heat
P	Pressure
V	Volume

CHAPTER 1

INTRODUCTION

1.1 STATEMENT AND SIGNIFICANCE OF THE PROBLEM

In modern educational institutions, students frequently require timely access to information and assistance navigating complex campus environments. Traditional support mechanisms, such as static information boards or staffed reception points, are not always available and often lack the flexibility to address the diverse and dynamic needs of individual students efficiently. This gap highlights the need for a more responsive and intelligent form of assistance that can operate continuously without reliance on human availability.

Recent advancements in artificial intelligence and robotics have made it increasingly feasible to deploy interactive robotic assistants capable of filling this role. Such systems can provide real-time navigation support, respond to a wide range of student queries, and contribute to a more engaging and accessible campus experience. When equipped with appropriate AI technologies, a robotic assistant can operate autonomously, adapt to its environment, and deliver consistent, accurate information on demand.

The robotic assistant proposed in this thesis is designed to address these needs within a defined campus setting. The system integrates Simultaneous Localization and Mapping (SLAM) to enable autonomous indoor navigation, voice recognition to support natural hands-free interaction, and Retrieval-Augmented Generation (RAG) to produce precise responses grounded in curriculum-specific data. Together, these technologies form a unified platform intended to offer a seamless and intelligent support mechanism for students within the E12 Building at KMITL.

1.2 GOAL AND OBJECTIVE OF THE STUDY

- Design and integrate a mobile robotic platform equipped with LiDAR sensors to enable real-time obstacle detection and SLAM-based autonomous navigation within the E12 Building.
- Incorporate a Large Language Model to support natural, context-aware conversation, allowing the robot to engage with students in open and meaningful dialogue.
- Implement a Retrieval-Augmented Generation pipeline to enhance the model's responses with accurate, curriculum-aligned information drawn from a domain-specific knowledge base.
- Develop a robust data management system using MySQL for structured record storage and Milvus as a vector database, enabling efficient management of interaction logs, curriculum data, and conversation history.
- Integrate Speech-to-Text and Text-to-Speech pipelines to provide fully hands-free voice interaction, removing the need for manual input during operation.
- Design a user-friendly interface that remains accessible and intuitive for students across varying levels of technological familiarity.

- Optimise the system for real-time performance to ensure that navigation, speech processing, and response generation are executed with minimal latency, delivering a seamless interactive experience.

1.3 HYPOTHESIS

1.3.1 Autonomous Navigation and Obstacle Avoidance

Mobile robot should be integrated with LiDAR sensors and SLAM algorithms will be capable of performing accurate real-time localization and obstacle avoidance within the E12 Building environment. This is supported by the theoretical foundations of probabilistic robotics and Simultaneous Localization and Mapping (SLAM), which enable robots to construct and update maps while navigating dynamic environments.

1.3.2 Effectiveness of Natural Language Interaction

The incorporation of a Large Language Model (LLM) will significantly improve the ability to engage in natural, context-aware conversations with users. This is based on advancements in transformer-based language models, which demonstrate strong capabilities in understanding and generating human-like dialogue.

1.3.3 Improved Response Accuracy through Retrieval-Augmented Generation (RAG)

Integrating a Retrieval-Augmented Generation pipeline will enhance the factual accuracy and relevance of responses compared to a standalone language model. This assumption is supported by information retrieval theory, where external knowledge sources improve response grounding and reduce hallucination.

1.3.4 Efficiency of Data Management System

The combination of MySQL for structured data storage and Milvus as a vector database will enable efficient storage, retrieval, and management of interaction logs and knowledge representations. This is based on database system principles and vector similarity search theory, which optimize both structured querying and semantic retrieval.

1.3.5 Usability of Voice-Based Interaction

Integrating Speech-to-Text (STT) and Text-to-Speech (TTS) systems will improve user accessibility and interaction efficiency by enabling hands-free communication. This aligns with human–computer interaction (HCI) principles, which emphasize natural and intuitive interfaces.

1.3.6 User Interface Accessibility and Acceptance

User-friendly interface will enhance usability and user satisfaction across individuals with varying levels of technological familiarity. This is supported by usability engineering and user-centered design theories.

1.3.7 Real-Time System Performance

Optimizing the integration of navigation, AI processing, and speech pipelines will result in low-latency system performance, enabling seamless real-time interaction. This is based on real-time systems theory, where system responsiveness is critical for effective human-robot interaction.

1.4 SCOPE OF THE STUDY

This study focuses on the development and evaluation of the robotic assistant within a defined operational environment. The robot is deployed exclusively on the 12th floor of the E12 Building, which serves as the primary testing and demonstration area. All navigation mapping, waypoint configuration, and environmental testing are conducted within this space, and the system is not designed or evaluated for operation beyond this boundary.

The target users of the system are students of the Robotics and Artificial Intelligence (RAI) programme. Interactions are therefore tailored to the needs and contexts relevant to this user group within the building environment. The robot is designed to support core interaction scenarios: providing navigation guidance to designated locations within the floor, and engaging in general conversation through the LLM-powered dialogue system.

1.5 PROCESS OF THE STUDY

The development of the robotic assistant follows a structured methodology comprising five sequential phases, spanning initial mechanical design through to final system evaluation.

1.5.1 Robot Mechanical Design

The first phase covers the mechanical design of the wheeled robot platform. The chassis is dimensioned to accommodate all onboard components, including the computing unit, sensors, battery, and drive system, while maintaining a stable centre of gravity suited to flat indoor surfaces.

1.5.2 Motor Selection and Placement

The second phase addresses motor selection and placement. Drive motors are chosen based on payload capacity, target operational speed, and manoeuvrability requirements. Motor controllers are configured to communicate with the main computing unit through the ROS 2 middleware framework.

1.5.3 Control System Design

The third phase involves the design of the control system architecture. This phase integrates the ROS 2 navigation stack, LiDAR-based SLAM for environment mapping and localisation, and AMCL for real-time pose estimation. The software layer connects perception, planning, and actuation subsystems through ROS 2 topics and services.

1.5.4 Navigation and Interaction

The fourth phase covers the development of the navigation and interaction pipelines. The conversational system couples the LLM with a RAG module supported by a vector database containing curriculum and building information. Speech-to-Text and Text-to-Speech components are integrated for hands-free voice interaction, and a face detection module enables the robot to proactively identify and approach visitors.

1.5.5 Testing and Evaluation

The fifth and final phase encompasses system testing and evaluation, conducted within the 12th floor of the E12 Building under realistic operating conditions. Evaluation criteria include navigation accuracy, speech recognition performance, response relevance, and user satisfaction, assessed through structured trials with RAI students.

1.6 ASSUMPTIONS

1.6.1 Hardware Capability

The onboard computing resources are sufficient to handle real-time processing tasks, including navigation, speech processing, and AI inference, within acceptable latency limits.

1.6.2 Power Availability

The robot's battery capacity and power management system are adequate to support continuous operation for the duration of testing.

1.6.3 Environmental Conditions

The E12 Building environment is semi-structured and does not undergo significant or unpredictable changes during operation. Static features such as walls, corridors, and major obstacles are considered stable, allowing reliable SLAM-based mapping and localization.

1.6.4 Network Connectivity

a stable network connection is available when required, particularly for accessing cloud-based services such as Large Language Models, communication, or external knowledge sources.

1.7 LIMITATION OF THE STUDY

- The system may require periodic updates to improve interaction accuracy.
- Budget constraints may limit the use of high-quality components, affecting overall performance.
- The robot's performance may be affected by environmental factors such as lighting conditions, noise levels, and physical obstacles, which could impact navigation and interaction quality.

1.7.1 Technical Constraints

- **Hardware Limitations**
  - **Computational Power:** The mini-PC must handle AI and LLM processing efficiently without excessive heat generation or power consumption.
  - **Battery Life:** Energy-efficient motors, sensors, and AI processing are required to maximize operational time.
  - **Mechanical Complexity:** The robot must ensure smooth and natural movement with a robust mobile base, including actuators and linkages.
  - **Sensor Accuracy:** High-resolution cameras and microphones are necessary for effective interaction while balancing cost and power consumption.
  - **Weight Distribution:** A well-balanced design ensures stability during movement and interaction.
- **Software and AI Limitations**
  - **Real-Time Processing:** Delays in speech recognition or posture analysis can affect user experience.

- **LLM Integration:** Requires fine-tuning and fast response times while maintaining contextual understanding.
- **Navigation and Mapping:** SLAM and obstacle detection algorithms are required for safe autonomous movement.
- **Robustness in Unstructured Environments:** The system must adapt to varying lighting, noise, and terrain conditions.

1.7.1.1 Non-Technical Design Constraints

- User Acceptance and Adaptability

  The robot must be designed for ease of use, ensuring that individuals of different technological backgrounds can interact with it effectively.

- Privacy and Data Security

  Handling user interactions requires adherence to data protection policies to maintain confidentiality and security.

- Cost and Scalability

  The system should be cost-effective for widespread adoption, balancing hardware affordability with advanced AI capabilities.

- Regulatory and Ethical Considerations

  Compliance with safety regulations and ethical AI deployment guidelines is essential to prevent bias, ensure transparency, and promote responsible AI usage.

1.8 DEFINITIONS

Abbreviation	Full Term	Definition
AMCL	Adaptive Monte Carlo Localization	Probabilistic localization algorithm for ROS-based autonomous navigation.
API	Application Programming Interface	Rules and protocols enabling software applications to communicate and exchange data.
ASR	Automatic Speech Recognition	Technology that converts spoken speech into written text.
LiDAR	Light Detection and Ranging	Remote sensing sensor that measures distances using laser pulses.
LLM	Large Language Model	AI model trained on large text datasets for understanding and generating human language.
NLP	Natural Language Processing	Subfield of AI concerned with enabling computers to understand human language.
RAG	Retrieval-Augmented Generation	Technique that enhances LLM outputs by retrieving relevant external information.
ROS	Robot Operating System	Open-source middleware framework for developing robot software.
SLAM	Simultaneous Localization and Mapping	Computational technique allowing robots to map unknown environments while tracking position.
STT	Speech-to-Text	Process of converting spoken audio signals into written text.
TTS	Text-to-Speech	Speech synthesis technology that converts written text into audible output.

CHAPTER 2

LITERATURE REVIEW

2.1 MECHANICAL DESIGN

This section covers the mechanical aspects of the robotic assistant's design, including chassis, mobility systems, and structural components.

2.1.1 Torque

Torque is a measure of the rotational force applied to an object, calculated as the product of the force and the distance from the pivot point. In robotics, torque is crucial for determining the robot's ability to perform tasks that require rotation or movement against resistance. The required torque for a robotic joint can be calculated using the formula:

$$T = Fxr \tag{2.1}$$

Where: - T is the torque (in Newton-meters, Nm) - F is the force applied (in Newtons, N) - r is the distance from the pivot point to the point where the force is

- Torque Calculation for Robot Arm Motors For robotic arms, torque is required to lift and manipulate objects. The torque needed is influenced by the mass of the object, the length of the arm, and gravitational force. The torque can be calculated using the formula:

$$T = mxgxl \tag{2.2}$$

Where: - m is the mass of the object being lifted (in kilograms, kg) - g is the acceleration due to gravity (approximately 9.81 m/s²) - l is the length of the arm from the pivot point to the center (in meters, m) If the arm consists of multiple segments, the total torque required is the sum of the torques from each segment:

$$T_{total} = \sum_{i=1}^{n} (m_i x g x l_i) \tag{2.3}$$

Where:

- n is the number of segments in the arm

- m_i is the mass of the object or segment at position i

- l_i is the length from the pivot point to the center of mass of segment i

- m_i and l_i correspond to the mass and length of each segment of the arm, including the object being manipulated. This calculation ensures that the motors selected for the robotic arm can provide sufficient torque to perform the desired tasks effectively.

- Torque Calculation for Mobile Robot Wheels For mobile robots, torque is required to overcome friction and

move the robot. The torque needed can be calculated using the formula:

$$T = (MxaxF_rxF_i)xr \tag{2.4}$$

Where: - M is the mass of the robot (in kilograms, kg) - a is the acceleration of the robot (in meters per second squared, m/s²) - F_r is the rolling resistance coefficient (dimensionless) - F_i is the incline factor (dimensionless, accounts for slope) - r is the radius of the wheel (in meters, m) Since frictional losses and real-world conditions can impact performance, the actual motor torque should be selected with a safety margin, typically 1.5 to 2 times the calculated torque.

2.1.2 Kinematics

Kinematics is the study of motion without consideration of the forces that cause it. In mobile robotics, kinematics describes the geometric relationship between actuator commands and the resulting motion of the robot in its environment. Understanding robot kinematics is essential for trajectory planning, odometry-based position estimation, and the design of motion control systems.

The robot employed in this project uses a differential drive configuration, in which two wheels are independently actuated on a shared axle. This configuration is widely adopted in indoor mobile robots due to its mechanical simplicity and ease of control. The instantaneous linear velocity v and angular velocity ω of a differential drive robot are derived from the individual wheel velocities as follows:

$$v = \frac{v_R + v_L}{2} \tag{2.5}$$

$$\omega = \frac{v_R - v_L}{d} \tag{2.6}$$

Where: - v_R is the velocity of the right wheel (in meters per second, m/s) - v_L is the velocity of the left wheel (in meters per second, m/s) - d is the distance between the two wheels, known as the wheel base (in meters, m)

The robot's pose in a two-dimensional plane is described by the state (x, y, θ), where (x, y) denotes the position and θ is the heading angle. The time derivatives of the pose components are given by:

$$\dot{x} = v\cos\theta, \quad \dot{y} = v\sin\theta, \quad \dot{\theta} = \omega \tag{2.7}$$

In practice, the robot's pose is continuously estimated through odometry by integrating these kinematic equations using wheel encoder measurements. However, integration over time accumulates errors introduced by wheel slip, encoder resolution limitations, and surface irregularities, resulting in odometry drift. To compensate for this drift, localisation algorithms such as Adaptive Monte Carlo Localisation (AMCL) fuse odometry data with external sensor observations, such as LiDAR scan matching, to maintain an accurate and consistent pose estimate during autonomous operation.

2.2 ELECTRICAL DESIGN

This section covers the electrical systems, power management, and hardware integration aspects of the robot.

2.2.1 Power Supply Architecture and Electrical Safety

The power supply architecture of a mobile robotic system determines how electrical energy is arranged, distributed, and protected throughout the platform. In addition to supplying the required voltage and current to each subsystem, the electrical design must also ensure safe operation under normal and fault conditions. For this reason, both connection configuration and protection devices are important considerations in robotic electrical systems.

In electrical design, the arrangement of components can be classified into series and parallel connections. These two connection types have different electrical characteristics and are used for different purposes in robotic systems.

2.2.1.1 Series and Parallel Connection

A series connection is formed when components are connected in a single continuous path, causing the same current to flow through every element. In this arrangement, the total voltage across the circuit is equal to the sum of the voltage drops across each component to increase the voltage. The total voltage of a series-connected source can be expressed as:

$$V_{total} = \sum_{i=1}^{n} (V_i) \tag{2.8}$$

Where each V_i is the voltage of an individual component in the series.

A parallel connection provides multiple paths for current flow. In this arrangement, each branch receives the same supply voltage, while the total current is shared among the loads according to their individual requirements. The total current in a parallel circuit is given by:

$$I_{total} = \sum_{i=1}^{n} (I_i) \tag{2.9}$$

Where each I_i is the current drawn by an individual component in the parallel arrangement

2.2.1.2 Electrical Safety Considerations

Electrical safety considerations are important in robotic systems to prevent damage caused by reverse polarity, overcurrent, electrical transients, and hazardous operating conditions. Diodes are commonly used for reverse polarity protection and flyback suppression in inductive circuits. Circuit breakers are used to disconnect the power path when abnormal current is detected, protecting the system from overheating and electrical faults. In addition, an emergency stop (EMO) provides a manual method for immediately shutting down robot operation during unsafe situations. Proper wiring, grounding, and component selection further contribute to the safe and reliable operation of the robotic platform.

2.2.2 DC to DC Buck Converter

A DC to DC buck converter is a type of switching voltage regulator used to reduce a higher direct current (DC) input voltage to a lower regulated DC output voltage. It is widely used in embedded systems, portable electronics, and mobile robotic platforms because many electronic components operate at voltage levels lower than the main battery supply. In robotic applications, a buck converter is especially important for powering low-voltage devices such as microcontrollers, sensors, single-board computers, and communication modules from a higher-voltage battery source. The operation of a buck converter is based on the controlled switching of a semiconductor device, typically a transistor or MOSFET, together with an inductor, diode or synchronous switch, and output capacitor. When the switch is turned on, energy from the input source is transferred to the inductor and load. During this period, the inductor stores energy in its magnetic field. When the switch is turned off, the inductor releases its stored energy to the load through the freewheeling path, allowing the output current to continue flowing. The capacitor helps smooth the output voltage by reducing voltage ripple caused by the switching action. The output voltage of an ideal buck converter can be expressed as:

$$V_{out} = DxV_{in} \tag{2.10}$$

Where: - V_{out} is the output voltage (in volts, V)

- D is the duty cycle (a value between 0 and 1)

- V_{in} is the input voltage (in volts, V)

In practical circuits, the output voltage is also influenced by component losses, switching frequency, load current, and internal resistance of the converter elements. Therefore, real buck converters are designed with feedback control mechanisms to maintain a stable output voltage even when the input voltage or load conditions vary. This voltage regulation characteristic is essential in robotic systems, where electronic modules may be sensitive to voltage fluctuations.

The efficiency of a converter is defined as the ratio of output power to input power, expressed as a percentage:

$$\eta = \left(\frac{P_{out}}{P_{in}}\right)x100\% \tag{2.11}$$

Where: - η is the efficiency of the converter (in percentage)

- P_{out} is the output power delivered to the load (in watts, W)

- P_{in} is the input power drawn from the source (in watts, W)

In summary, the DC to DC buck converter is a key component in modern robotic electrical systems because it provides efficient and regulated voltage conversion from the main battery supply to lower-voltage electronic subsystems. Its high efficiency, controllable output, and suitability for battery-powered applications make it an essential part of the electrical design of mobile robots.

2.2.3 Power Consumption Analysis

Power consumption is a fundamental consideration in the electrical design of mobile robotic systems because it directly affects operating time, system stability, component selection, and overall efficiency.

A mobile robot typically consists of multiple electrical and electronic subsystems, such as embedded controllers, onboard computers, sensors, motor drivers, communication modules, and audio or display interfaces. Each subsystem consumes electrical power according to its voltage and current requirements. Therefore, understanding and estimating the total power demand of the robot is essential during the design stage.

In electrical systems, power is defined as the rate at which electrical energy is consumed by a load. The basic relationship between voltage, current, and power is expressed by:

$$P = VxI \tag{2.12}$$

Where:

- P is the power consumption (in watts, W)
- V is the voltage supplied to the component (in volts, V)
- I is the current drawn by the component (in amperes, A)

The total power consumption of a robotic platform can be expressed as the sum of the power consumed by all electrical loads:

$$P_{total} = \sum_{i=1}^{n} (P_i) \tag{2.13}$$

Where: - P_i is the power consumption of each subsystem i

1. Static Power Consumption: Static power consumption refers to the relatively constant power drawn by components that operate continuously, such as microcontrollers, onboard computers, LiDAR sensors, and communication modules.
2. Dynamic Power Consumption: Dynamic power consumption refers to the variable power demand caused by components whose load changes during operation, particularly motors and actuators. For example, a robot may consume significantly more power while accelerating, turning, or carrying payloads than when it is idle or moving at constant speed.

The operating time of a battery-powered robot can be roughly estimated using the battery energy capacity and the average system power consumption. Battery energy is commonly expressed in watt-hours (Wh), which can be calculated as:

$$E = CxV \tag{2.14}$$

Where:

- E is the energy capacity of the battery (in watt-hours, Wh)
- C is the battery capacity (in ampere-hours, Ah)
- V is the nominal voltage of the battery (in volts, V)

The estimated runtime of the robot can then be calculated as:

$$T = \frac{E}{P_{total}} \tag{2.15}$$

Where: - T is the estimated runtime (in hours) - P_{total} is the total power consumption of the robot (in watts)

In summary, power consumption is a key parameter in the design of mobile robotic systems. It influences battery selection, operating time, voltage regulation, thermal performance, and system reliability. A

thorough understanding of voltage, current, and power relationships enables designers to estimate energy demand accurately and develop a stable electrical architecture suitable for continuous robotic operation.

2.2.4 Electrical Interconnection and Power Distribution

Electrical interconnection and power distribution are essential aspects of robotic electrical design because they determine how electrical energy is delivered from the power source to each subsystem in a safe, stable, and efficient manner. In a mobile robot, multiple components such as embedded controllers, onboard computers, sensors, motor drivers, audio modules, and communication devices operate simultaneously, often with different voltage and current requirements. Therefore, an appropriate electrical interconnection design is necessary to ensure that each subsystem receives the correct supply voltage and sufficient current for reliable operation.

Power distribution must also be designed to minimize undesirable effects such as voltage drop, electrical noise, and overheating. Voltage drop occurs when current flows through wires or connectors with finite resistance, reducing the effective voltage reaching the load. This can become significant in systems with long wires or high-current devices such as motors. To reduce voltage drop, designers typically use conductors with appropriate wire gauge, minimize unnecessary cable length, and ensure good-quality electrical connections. Stable voltage delivery is especially important for sensitive electronics such as microcontrollers, sensors, and onboard computers, which may malfunction if the supply voltage falls below their required operating range.

Electrical noise is another major concern in robotic systems, especially when low-power digital electronics and high-power actuators share the same power source. Motors, switching converters, and communication devices can introduce noise and transient disturbances into the power line. These disturbances may degrade sensor readings, interfere with communication signals, or affect processor stability. For this reason, electrical interconnection design often includes filtering capacitors, decoupling capacitors, proper grounding strategies, and physical separation between power and signal wiring. In some cases, sensitive subsystems are provided with isolated or separately regulated power lines to improve reliability. Thermal management is also a critical aspect of power distribution. High-current components such as motor drivers and voltage regulators can generate significant heat during operation. Proper heat dissipation through heatsinks, ventilation, or active cooling may be necessary to prevent overheating and ensure long-term reliability.

In addition, the power distribution architecture must consider the different operational characteristics of system components. High-current devices such as motors may produce sudden current surges during startup or direction changes, while computing units and sensors typically require stable, low-noise power supplies. Therefore, the electrical design should allocate power paths according to the characteristics of each load. Separating high-power and low-power branches can improve system stability and reduce interference between subsystems.

In summary, electrical interconnection and power distribution form the foundation of a reliable robotic electrical system. Proper distribution of power from the battery to all subsystems ensures that each component receives the required voltage and current while maintaining overall safety, stability, and efficiency.

2.3 SOFTWARE DESIGN

2.3.1 Mobile Robot

2.3.1.1 ROS 2 (Robot Operating System 2)

ROS 2 is an open-source robotic middleware framework that has become the de facto standard for robot software development. Designed from the ground up to overcome limitations of the original ROS, it provides comprehensive support for modern robotic systems, including multi-robot teams, resource-constrained embedded platforms, real-time control systems, and operation across unreliable networks. ROS 2 establishes well-defined design patterns and architectural conventions for structuring complex robotic software, supporting modularity, scalability, and maintainability in autonomous systems. It targets production-grade deployment environments where reliability and determinism are critical, combining distributed middleware capabilities with tools and libraries that streamline development of sophisticated control, perception, and communication subsystems. For this project, ROS 2 serves as the core integration middleware connecting all robot subsystems.

2.3.1.2 ROS Navigation Stack

The ROS Navigation Stack provides a comprehensive framework for autonomous mobile robot navigation, accepting odometry data and sensor stream inputs and outputting velocity commands to the robot's motor controller. This creates a clean abstraction that separates high-level path planning logic from low-level motor actuation. The Navigation Stack requires correct configuration of several components: ROS middleware access, a tf transform tree defining coordinate frame relationships between hardware components, standardised sensor data publication using ROS message types, and tuned costmap and planner parameters reflecting the robot's geometry and dynamics. This modular architecture enables robust navigation in diverse environments while supporting independent testing and development of individual components.

2.3.1.3 AMCL (Adaptive Monte Carlo Localization)

Adaptive Monte Carlo Localization (AMCL) is the primary localization package used within the ROS ecosystem for autonomous navigation tasks. The algorithm employs a particle filter approach, maintaining a probabilistic distribution of candidate robot poses and iteratively refining this distribution as new sensor observations and odometry measurements become available. AMCL is specifically optimised for use with 2D laser rangefinders and pre-built occupancy grid maps, making it well-suited to structured indoor environments.

It is important to note that AMCL is subject to specific operational constraints. The algorithm assumes a fixed geometric relationship between the laser sensor and the robot base; any movement of the laser relative to the base frame violates this assumption and can degrade localization accuracy significantly. Furthermore, AMCL requires continuous odometry input to propagate the particle filter over time. Odometry estimates may be derived from multiple complementary sensor sources, including wheel encoders and inertial measurement units (IMUs), which together provide more robust position and orientation estimates than either source alone.

2.3.2 Database Management

2.3.2.1 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) is a technique that enhances large language model output quality by incorporating an external knowledge retrieval step before response generation. Rather than relying solely on patterns in the model's training parameters, RAG queries a domain-specific knowledge base at inference time, retrieving contextually relevant information to augment the model's context.

RAG offers cost-effective extension of LLM capabilities to specialised domains without expensive model retraining. By grounding responses in retrieved, current information, RAG substantially reduces factual hallucinations and improves relevance and accuracy. In this implementation, RAG forms the backbone of the knowledge retrieval subsystem, enabling contextually appropriate responses to user queries about the operating environment and domain-specific topics.

2.3.2.2 Vector Database

A vector database stores data as dense numerical vectors (embeddings) rather than conventional row-column structures, enabling similarity-based retrieval over large unstructured data collections including text, images, and audio. Vector databases support approximate nearest-neighbour search rather than exact-match queries, enabling retrieval of semantically related records even with differing query wording. This functionality is essential for RAG systems where semantic relevance rather than lexical overlap determines retrieval quality.

2.3.2.3 Embeddings

Embeddings are dense, fixed-dimensional numerical representations of objects — including text, images, and audio — produced by machine learning models trained to capture semantic and contextual relationships. Objects that are semantically similar are mapped to nearby points in the embedding space, enabling distance-based similarity measures to serve as a proxy for meaning. In this system, text embeddings represent both stored knowledge base documents and incoming user queries, enabling the vector database to identify and retrieve the most relevant passages for augmenting the LLM's response.

2.3.2.4 Milvus

Milvus is an open-source vector database platform specifically engineered for large-scale similarity search on massive embedding collections. Designed with production deployment in mind, Milvus provides highly optimised indexing structures and query algorithms that enable fast, accurate retrieval of semantically similar vectors from datasets containing millions or billions of embeddings. Unlike traditional relational databases, Milvus is purpose-built for vector search operations, supporting multiple similarity metrics including Euclidean distance, cosine similarity, and inner product.

Key features of Milvus include exceptional scalability through distributed architectures that allow horizontal expansion, high-performance approximate nearest-neighbour search capabilities that retrieve relevant results with minimal latency even on very large datasets, and flexible indexing strategies optimised for different trade-offs between search accuracy and computational efficiency. In this implementation, Milvus serves as the underlying vector database infrastructure for the RAG pipeline, enabling rapid semantic retrieval of domain-relevant information from the knowledge base to augment LLM responses.

2.3.2.5 SQL (Structured Query Language)

SQL is a standardised declarative programming language for defining, manipulating, and retrieving data in relational database management systems (RDBMS). Data is organised in tables with rows representing records and columns representing typed attributes. SQL provides comprehensive operations for schema creation, record insertion and updates, referential integrity enforcement, and complex multi-table queries through joins and aggregations. Query optimisation and index management facilities ensure retrieval performance at scale. In this system, SQL maintains structured records of user interactions, profiles, and system logs, complementing the vector database layer for semantic retrieval.

2.3.3 Artificial Intelligence

2.3.3.1 Face Recognition

Facial recognition is a biometric identification technology that authenticates or identifies individuals by analysing distinctive geometric and photometric features of the human face. It detects facial regions, generates compact feature representations, and compares them against stored facial templates. The system automatically identifies returning users and personalises interactions based on stored preferences and interaction history. This capability enhances human-robot interaction naturalness and continuity by eliminating explicit identification steps. For the robotic assistant, facial recognition enables context-aware, personalized engagement with individual users.

Modern face recognition systems are implemented as a multi-stage pipeline comprising face detection, multi-object tracking, frontal face filtering, and embedding-based identity matching. A face detector first identifies the locations of faces within each video frame, producing bounding boxes together with facial landmark points. Multi-object tracking then associates detections across consecutive frames, assigning a persistent track identifier to each individual to maintain identity continuity through brief occlusions and periods of low detection confidence. A head pose estimator computes the yaw, pitch, and roll angles of each tracked face from the landmark coordinates; only faces satisfying a frontal orientation constraint are eligible for recognition, preventing degraded embedding quality caused by profile or extreme off-angle views. Eligible face crops are passed to a recognition embedding model that produces a compact, fixed-dimensional numerical representation of facial appearance. Identity is then determined by comparing the query embedding against a gallery of stored enrolment embeddings using a similarity metric such as cosine similarity, returning a known identity when the similarity exceeds a configurable threshold, or assigning an unknown label otherwise. This multi-stage design balances computational efficiency with recognition accuracy in real-time robotic deployment scenarios.

2.3.3.2 Natural Language Processing (NLP)

Natural Language Processing (NLP) is a multidisciplinary field enabling computational systems to understand, interpret, and generate human language in written and spoken forms. It combines rule-based linguistics, statistical models, and deep learning architectures for tasks including parsing, sentiment analysis, intent detection, and machine translation. NLP systems capture not only surface language form but also deeper semantic content, including speaker intent, implied meaning, and emotional tone. This nuanced understanding enables the robotic assistant to engage in natural, contextually appropriate dialogue, especially in open-domain settings with ambiguous

or colloquial queries.

2.3.3.3 Large Language Model (LLM)

A Large Language Model (LLM) is a class of artificial intelligence system trained on massive corpora of text data — often comprising hundreds of billions of tokens — to acquire broad and deep competencies in understanding and generating human language. LLMs are built on the transformer neural network architecture, which leverages self-attention mechanisms to capture long-range dependencies between tokens and model the statistical structure of language at multiple levels of abstraction.

Through large-scale pre-training followed by task-specific fine-tuning or instruction alignment, LLMs develop the capacity to perform a diverse array of language tasks — including question answering, summarization, translation, and code generation — without requiring explicit task-specific training data for each application. In this system, the LLM serves as the central reasoning and response generation component of the conversational AI subsystem, processing retrieved contextual information from the RAG pipeline to produce coherent, factually grounded responses to user queries.

2.3.3.4 Text-to-Speech (TTS)

Text-to-Speech (TTS) is a speech synthesis technology that converts written text into natural-sounding spoken audio, enabling AI systems to communicate verbally with users without requiring pre-recorded audio assets. Modern TTS systems are built on deep learning architectures — including neural vocoders and sequence-to-sequence models — that generate speech with dynamic, context-sensitive variations in prosody, pitch, speaking rate, and pronunciation. These characteristics produce output that closely approximates natural human speech patterns, significantly improving comprehensibility and user engagement compared with earlier rule-based synthesis methods.

TTS technology has found widespread application in accessibility tools for visually impaired users, virtual assistants, and interactive voice response systems. For the robotic assistant, TTS serves as the primary output modality for verbal communication, enabling the robot to deliver responses audibly in environments where visual screen interaction is impractical or undesirable.

2.3.3.5 Automatic Speech Recognition (ASR)

Automatic Speech Recognition (ASR) is a technology that processes raw acoustic speech signals and converts them into machine-readable text, forming the primary input pathway for voice-driven human-machine interaction. Contemporary ASR systems employ end-to-end deep learning architectures — including recurrent neural networks and transformer-based models — trained on large-scale speech corpora to achieve high transcription accuracy across a range of speakers, accents, and acoustic conditions.

Advanced ASR implementations integrate Natural Language Processing modules downstream of the transcription stage, enabling the system to infer the user's intent and semantic meaning from the recognised text in addition to producing a verbatim transcript. ASR accuracy is sensitive to several environmental factors, including background noise levels, microphone quality, and speaker proximity; consequently, robust noise handling and signal preprocessing are important design considerations for deployment in real-world robotic environments. Despite these challenges, modern ASR systems are capable of achieving near real-time transcription latency, enabling fluid conversational interactions between the user and the robotic assistant.

2.3.3.6 Voice Activity Detection (VAD)

Voice Activity Detection (VAD) is a technology that automatically identifies the presence or absence of human speech within an audio signal. It serves as a critical preprocessing stage in speech processing pipelines, enabling downstream modules such as automatic speech recognition to process only speech-containing segments while discarding silence, background noise, and non-speech acoustic events. By filtering out non-speech audio before ASR inference, VAD reduces unnecessary computational load and prevents the hallucination of spurious transcribed text from ambient sounds or environmental noise.

Traditional VAD approaches relied on energy thresholding or zero-crossing rate estimation, which are sensitive to noise and require manual parameter tuning for each deployment environment. Contemporary VAD systems employ deep neural networks trained on large and diverse speech corpora to output a probability score for each audio frame, indicating the likelihood that the frame contains speech. Frames with probability values exceeding a configurable threshold are classified as speech and forwarded to the speech recognition backend, while frames below the threshold are discarded.

In robotic systems deployed in indoor environments, VAD is especially important for suppressing hallucination loops in neural ASR models, which can generate repeated words or phrases when continuously presented with background noise or silence. A two-tier VAD pipeline — combining a pre-emphasis filter to amplify high-frequency speech components with a neural probability-based VAD model — provides robust and reliable speech detection in acoustically challenging conditions without the need for keyword or wake-word detection mechanisms.

2.3.3.7 Multi-Object Tracking

Multi-object tracking (MOT) is the computer vision problem of jointly detecting and maintaining the identities of multiple objects across a sequence of video frames. In robotic perception systems, tracking is used to associate detections across time, enabling the system to follow individual persons or objects even when they temporarily leave the field of view or are occluded by other scene elements.

Contemporary tracking algorithms typically follow the tracking-by-detection paradigm, in which a detector is applied independently to each frame and a subsequent data association step links detections to existing tracks using geometric or appearance-based cues. High-performance tracking methods such as ByteTracker extend conventional association by incorporating low-confidence detection boxes in a secondary matching pass, recovering objects that are partially occluded or temporarily invisible and maintaining more stable trajectories across challenging sequences. Each tracked individual is assigned a persistent track identifier that remains consistent across frames, enabling downstream processes — such as face recognition and interaction management — to apply per-identity cooldowns, accumulate recognition evidence over multiple frames, and gate interaction triggers to prevent false activations from brief or transient detections.

CHAPTER 3

METHODOLOGY

3.1 System Overview

The Satu robotic assistant is implemented as a distributed embedded architecture consisting of four computational nodes: a central AI Brain server, an NVIDIA Jetson Orin Nano vision module, and two Raspberry Pi 5 edge devices. This distributed design balances computational workload while minimising latency across perception, reasoning, and actuation subsystems.

The AI Brain server acts as the core intelligence layer, operating a FastAPI-based system that manages the entire interaction pipeline. It integrates large language model (LLM) inference via Ollama, a Retrieval-Augmented Generation (RAG) pipeline, speech-to-text (STT) and text-to-speech (TTS), and database services. A hybrid storage system is employed, combining Milvus for vector-based semantic retrieval and MySQL for structured relational data. Additionally, SQLite is used for lightweight conversation logging and system monitoring.

The Jetson Orin Nano module is responsible for real-time face detection and recognition, leveraging GPU acceleration through TensorRT-optimised ONNX models to perform biometric identification of registered users with low latency.

The first Raspberry Pi 5 functions as the audio interface and coordination node, handling microphone input, performing voice activity detection and on-device speech transcription, and playing back synthesised speech responses received from the server. It also hosts the robot's expressive face display, rendered on an HDMI screen using an Electron.js application. The second Raspberry Pi 5 operates as the mobility controller, executing ROS2-based navigation, including path planning, wheel control, and obstacle avoidance.

Component	Specification
AI Brain server	Ubuntu 22.04 LTS, NVIDIA GPU
Raspberry Pi 5 #1 (audio + display)	Raspberry Pi OS (64-bit)
Raspberry Pi 5 #2 (navigation)	Raspberry Pi OS (64-bit)
Jetson Orin Nano	NVIDIA Jetson Orin Nano (8 GB)
LiDAR sensor	RPLiDAR A1 (2D, 360°)
Drive controller	iREDCr differential drive
Navigation middleware	ROS2 Jazzy Jalisco + Nav2
LLM inference	Ollama (Typhoon2-70B, Q5_K_M GGUF)
Vector database	Milvus v2.6
Relational database	MySQL + SQLite
Audio processing	Typhoon2-Audio 8B (STT + TTS sidecar)
Face recognition	SCRFD + ArcFace (ONNX / TensorRT)

Table 3.1 System Components and Specifications

3.1.1 Distributed Node Architecture

The four computational nodes are connected over a local area network and communicate via a combination of persistent WebSocket channels and stateless HTTP requests. The network topology is intentionally

hub-and-spoke: the AI Brain server acts as the central coordinatorAll edge nodes communicate solely with this server rather than directly with each other, except for a dedicated Jetson-to–Raspberry Pi 5 WebSocket connection that enables low-latency transmission of face detection events.

Host	IP Address	Port	Protocol	Service
AI Brain server	10.100.16.22	8000	HTTP	Main FastAPI service
AI Brain server	10.100.16.22	8001	HTTP	Typhoon2-Audio sidecar
AI Brain server	10.100.16.22	19530	gRPC	Milvus vector database
Jetson Orin Nano	192.168.1.10	9999	TCP	Remote shutdown listener
Pi 5 #1 (audio)	10.26.9.196	8765	WebSocket	Jetson detection receiver
Pi 5 #1 (audio)	10.26.9.196	8766	HTTP	RaspiReceive FastAPI
Pi 5 #1 (display)	10.26.9.196	7000	HTTP	Face emotion API
Pi 5 #1 (internal)	127.0.0.1	8768	WebSocket	Face UI IPC bridge
Pi 5 #2 (navigation)	10.26.9.196	8767	HTTP	ROS2 navigation bridge

Table 3.2 Network Endpoints and Communication Protocols

Figure 3.1 illustrates the distributed node topology and the primary communication links between all four computational nodes.

Figure 3.1 System Overview: Distributed Node Architecture and Communication Pathways

3.1.2 End-to-End Interaction Pipeline

The complete pipeline for a single human-robot interaction follows a linear sequence of processing stages:

1. The Jetson Orin Nano continuously captures camera frames and applies face detection and recognition. Upon identifying a student or an unidentified visitor, it transmits a detection event as a JSON message over the persistent WebSocket to Pi 5 #1.
2. Pi 5 #1 receives the detection event and simultaneously records microphone audio. A Voice Activity Detection model gates speech capture, and a speech recognition model transcribes the captured segment.

3. The coordination logic on Pi 5 #1 combines the detection event with the transcription result and dispatches a single JSON payload to the AI Brain server via an HTTP POST request.

4. The AI Brain server executes the full inference pipeline: grammar correction, query routing, RAG retrieval, LLM response generation, and TTS synthesis.

5. The server dispatches responses asynchronously: synthesised audio (WAV) to Pi 5 #1 for playback, a microphone activation flag (**/set_active**) to the audio controller, and when required, a navigation command to Pi 5 #2. Face emotion transitions are managed locally by Pi 5 #1 based on its own audio playback lifecycle.

3.2 Electrical System Design

3.2.1 Power Supply Architecture

The overall power supply architecture of the robot system is illustrated in Figure 3.2. The design integrates multiple voltage regulators and distributes power to key subsystems including the Raspberry Pi, sensors, and actuators.

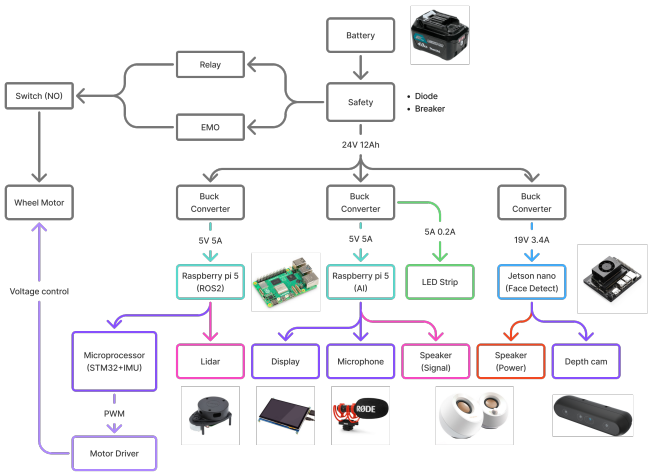

Figure 3.2 Overview of Power Supply Architecture

3.3 ROS System Architecture and Development

3.3.1 ROS Architecture Overview

The ROS2 system architecture in this project is designed to organize the robot software into multiple functional layers that work together to achieve autonomous navigation and human interaction. The overall system is structured in a layered pipeline, starting from the external AI server and progressing through the HTTP bridge, navigation manager, Nav2 stack, kinematics layer, and finally the motor controller.

In this architecture, high-level commands from the AI server are gradually converted into low-level motor actions. For example, navigation commands are first received via HTTP, then translated into ROS2 messages. These messages are processed by the navigation state machine, passed to the Nav2 stack for path planning, and then converted into wheel velocities using inverse kinematics. Finally, the computed motor speeds

are sent to the motor controller through the Modbus RTU protocol.

At the same time, sensor data such as encoder and LiDAR readings flow in the opposite direction to support decision-making. This bidirectional communication allows the robot to adjust its behavior based on both external commands and real-time environmental feedback.

3.3.2 Hardware Communication and Motor Control

The hardware communication layer is handled by the **iredcr_modbus_node** in the **ired_bringup** package. This node is responsible for communication between the ROS2 system and the embedded motor controller using the Modbus RTU protocol over a serial interface.

The communication is configured at 115200 baud via the **/dev/ttyiREDCr** port. The node runs on a 10 ms loop, performing two main operations in each cycle:

1. Writing motor speed commands to the controller
2. Reading feedback data from the controller

The feedback includes four wheel encoder values and six IMU measurements (linear acceleration and angular velocity). These values are then published as ROS2 topics for further processing.

To ensure safety, a timeout mechanism is implemented. If no velocity command is received within one second, all motor speeds are set to zero automatically. This prevents unintended movement in case of communication failure. Additionally, an encoder validation check is applied to avoid incorrect zero readings by keeping the previous valid value when necessary.

3.3.3 Omnidirectional Kinematics

The robot uses a four-wheel mecanum drive system, which allows movement in all directions without changing orientation.

Two ROS2 nodes are used to handle kinematics:

1. **Forward kinematics node:**This node calculates the robot's velocity based on encoder feedback. Wheel speeds are converted from RPM to meters per second, and then combined to compute linear velocities (x, y) and angular velocity (z). The result is published as a **TwistStamped** message.
2. **Inverse kinematics node:**This node performs the opposite process. It receives

3.3.4 IMU Processing and Odometry Estimatio

velocity commands from **/cmd_vel** and converts them into individual wheel speeds. These speeds are then sent to the motor controller.

3.3.5 IMU Processing and Odometry Estimation

The IMU data from the motor controller is processed by the imu_node. This node converts raw sensor values into a standard ROS2 IMU message format.

The orientation (roll, pitch, yaw) is calculated by integrating angular velocity over time. The results are published both as Euler angles and quaternion format.

The odom_node uses both the velocity from forward kinematics and the orientation from the IMU to estimate the robot's position. The position is updated every 10 ms by integrating velocity over time.

The computed odometry is published to **/odom_encoder** and used as the main motion esti-

mation for the navigation system. This encoder-based odometry is later combined with LiDAR-based estimation to improve accuracy.

3.3.6 LiDAR Integration and Scan Filtering

The robot uses an **RPLIDAR A1** sensor to detect its surroundings. The sensor is connected via serial communication and managed by the rplidar_ros package.

The LiDAR provides 360-degree scan data, which is processed and published as a LaserScan message. To improve data quality, a filtering process is applied.

A minimum distance filter (0.40 m) is used to remove points that come from the robot's own structure. The filtered scan is then published to scan_filtered and used for both navigation and localization.

3.3.7 Scan Matching Odometry

The system uses the ros2_laser_scan_matcher package to estimate motion based on LiDAR data. This method uses the Canonical Scan Matcher (CSM) algorithm.

The algorithm compares consecutive scans and calculates the robot's movement by minimizing the distance between matching points. The result is an additional odometry estimate published to **/odom**.

This method improves accuracy compared to encoder-only odometry, especially in environments with wheel slip.

3.3.8 Localisation and Navigation Stack

3.3.8.1 Map-Based Localisation with AMCL

The robot uses Adaptive Monte Carlo Localization (AMCL) to estimate its position on a known map.

AMCL uses a particle filter to represent possible robot positions. Each particle is updated based on LiDAR data, and over time, the particles converge to the correct position.

3.3.8.2 Auto-Localisation at Initial Position

To simplify operation, an automatic localization process is implemented.

At startup, the robot performs a 360-degree rotation while AMCL updates its estimate. This process runs for at least 15 seconds to ensure accuracy.

The robot executes one or more complete 360-degree rotation manoeuvres using the Nav2 spin behaviour, during which the LiDAR continuously sweeps all surrounding directions. As each scan is processed, AMCL matches the observed geometry against the stored map walls, causing the particle distribution to converge progressively toward the true robot pose.

3.3.8.3 Path Planning

Once localisation is established, the navigation system generates a collision-free path from the robot's current estimated pose to the target destination using the Nav2 stack.

The global planner computes an initial path over the occupancy grid map, while the local planner, implemented using the DWB controller, executes real-time trajectory optimisation within a rolling local costmap constructed from live LiDAR scans.

3.3.8.4 Stuck Detection and Recovery

During navigation, the system continuously monitors the robot's progress toward its goal by tracking the distance_remaining value provided in Nav2 action feedback at regular intervals.

If the robot does not move closer to the goal within 15 seconds, it is considered stuck. The system then:

1. Cancels the current goal
2. Performs re-localization
3. Retries the goal (up to 2 times)

This layered recovery strategy ensures that the robot can resume autonomous operation after transient failures without requiring external intervention.

3.3.9 Navigation State Machine

The navigation system is controlled by a finite state machine implemented in the Navigation-Manager node.

3.3.9.1 Idle and Stop State

The idle or stop state represents the condition in which the robot remains stationary to support human interaction processes while awaiting further navigation commands. In this state, the robot temporarily suspends all movement-related operations to allow external interaction modules, such as the AI-based communication system, to engage with users without interruption.

3.3.9.2 Autonomous Roaming State

In the autonomous roaming state, the robot performs exploratory movement within the environment without a predefined destination, allowing continuous operation when no specific task is assigned. This behaviour is implemented through a predefined set of waypoints, where the robot navigates sequentially between locations in a looping manner to maintain coverage of the environment. At each waypoint, the robot performs a short waiting process to observe its surroundings and allow potential interaction opportunities.

3.3.9.3 Goal-Directed Navigation State

When a target location is provided, the system transitions into the goal-directed navigation state, where it executes path planning and motion control to reach the specified destination. The robot continuously adjusts its movement based on localisation updates and environmental feedback to ensure accurate and safe navigation. In this project, the target goal is not generated autonomously but is obtained from the human interaction process, where the selected destination corresponds to predefined goal points already marked within the navigation map.

3.3.9.4 HTTP Bridge for External System Integration

The pi5_service node provides a REST API using FastAPI to connect the AI server with the ROS2 system.

The service listens on port 8767 and exposes two endpoints that correspond to the main interaction events defined in the system design. The POST /nav_state endpoint accepts a state integer and an optional destination codename, and the GET /health endpoint returns the current navigation state.

The node is launched alongside the NavigationManager using a single launch file, with ROS2 spinning in a background thread while the FastAPI server runs in the main thread. Upon receiving a navigation request, the service validates the state value and, for goal-directed commands, performs a dictionary lookup to translate the room codename to the corresponding ROS2 goal name. The translated command is then published as a string message on /navigation_command, which is consumed by the navigation state machine.

3.4 Software Design and Development

3.4.1 Software Architecture Overview

The system is structured into three tightly integrated layers: (1) a vector database for semantic retrieval, (2) a relational database for structured data management, and (3) an LLM integration module for conversational intelligence. These components work together to support real-time interaction, contextual reasoning, and personalised responses.

The five software repositories that constitute the system are assigned to their respective hardware nodes as follows.

- **JetsonVision** – Python application deployed on the NVIDIA Jetson Orin Nano. Responsible for real-time face detection, multi-face tracking, head pose estimation, and face recognition.
- **RaspiReceive** – Python asyncio application deployed on Pi 5 #1. Responsible for microphone capture, voice activity detection, speech transcription, detection-speech correlation, and audio playback.
- **baymax-face-ui** – Electron.js frontend combined with a Python FastAPI backend, deployed on Pi 5 #1. Renders the robot's emotional face on an HDMI display.
- **capstone_server** – Python FastAPI application deployed on the AI Brain server. Hosts the complete AI pipeline including grammar correction, RAG retrieval, LLM inference, TTS synthesis, and response dispatch.
- **Ros2_robot** – ROS2 Jazzy package suite deployed on Pi 5 #2. Provides hardware bring-up, sensor drivers, AMCL localisation, Nav2 path planning, and an HTTP bridge for external command injection.

3.4.2 Database Design

A hybrid database architecture is used to combine the advantages of both vector-based and relational storage systems. The vector database supports efficient semantic retrieval of unstructured data, while the relational database maintains data integrity and enables reliable transactional processing.

3.4.2.1 Milvus Vector Database

Milvus functions as the system's semantic memory layer, storing vector embeddings of unstructured data such as university information, curriculum materials, and timetable data. Instead of storing raw text, documents are transformed into 1024-dimensional dense vectors using the **BAAI/bge-m3** model, enabling efficient similarity-based retrieval.

Each collection is indexed using the **IVF_FLAT** method with **nlist** set to 128. During querying, an **nprobe** value of 10 is used, allowing approximate nearest-neighbor (ANN) search based on cosine similarity. The database is organised into four primary collections:

- **uni_info:** Contains seven entries describing campus buildings, facilities, zones, and departmental information within Building E-12 at KMITL. Serves as the default fallback retrieval source for campus navigation queries.
- **curriculum:** Derived from the Robotics and Artificial Intelligence Engineering programme specification document, containing 516 Thai-language chunk embeddings covering course descriptions, learning outcomes, prerequisites, and credit structures for Years 1 through 4.
- **time_table:** Constructed from an academic Excel timetable dataset. Each entry encodes a structured Thai sentence of the form (day, time slot, course name, room) to maximise retrieval precision, resulting in 128 entries.
- **conversation_memory:** A dynamic collection storing per-student LLM-generated conversation summaries, indexed by **student_id**. Supports filtered semantic retrieval, allowing the system to recall contextually relevant past interactions for a specific student during personalised prompt construction.

Semantic retrieval is performed using approximate nearest-neighbour search with an IVF_FLAT index and cosine similarity, significantly reducing response latency compared to traditional search methods.

3.4.2.2 MySQL Relational Database

MySQL manages structured data required for system operations. The schema enforces referential integrity and supports efficient querying of user-related information.

Key tables include:

- **Academic_Year:** Stores academic calendar information, including year boundaries used for tone adaptation in personalised prompts.
- **Students:** Maintains student identity records, including student identifier, full name and nickname in Thai and English, academic year, and enrolment status. This table is the authoritative source for student metadata injected into LLM prompts.
- **Face_Recognition_Data:** Stores biometric face encodings linked to student identities. Used by the Jetson Orin Nano enrolment tool to maintain the face embedding database.
- **ExcelTimetableData:** Contains structured timetable records for precise SQL-based schedule lookups, complementing the semantic retrieval provided by the **time_table** Milvus collection.

3.4.2.3 Database Synchronisation

Timetable data is synchronised across both systems: MySQL stores structured records, while Milvus stores corresponding embeddings for semantic retrieval. Updates are applied to both layers to maintain consistency between structured queries and conversational responses.

The **conversation_memory** collection is updated asynchronously at the end of each conversation session. The 8B LLM summarises the session transcript into a concise natural-language paragraph, which is then embedded and stored in Milvus tagged with the student's **student_id**. This fire-and-forget pattern prevents memory writes from blocking the main response pipeline.

3.4.3 LLM Integration Module

The LLM module serves as the conversational core, implementing a multi-route RAG pipeline that dynamically selects the appropriate data source for each query. The system uses a dual-model setup: a high-capacity 70B model for response generation and a lightweight 8B model for preprocessing tasks such as grammar correction and memory summarisation. Both models run via the Ollama inference server with GPU acceleration.

- **Main chatbot: llama3.1-typhoon2-70b-instruct** (Q5_K_M GGUF) – used for greeting generation and final response synthesis.
- **Fast assistant: llama3.1-typhoon2-8b-instruct** – used for grammar correction and end-of-session memory summarisation.
- **Audio sidecar: typhoon-ai/llama3.1-typhoon2-audio-8b-instruct** – a multimodal model deployed as an independent sidecar process on port 8001, handling both STT transcription and TTS synthesis.

3.4.3.1 Query Routing Strategy

User queries are classified using keyword-based intent detection and routed to one of six data sources:

- **Conversation Memory:** Retrieves the top-3 most semantically similar past conversation summaries from the **conversation_memory** Milvus collection, filtered by **student_id**. Provides long-term personalisation context.
- **Student Information:** Executes a structured SQL query against MySQL to retrieve the student's academic year and metadata. The **student_year** field defaults to 1 on failure to prevent pipeline interruption.
- **Timetable:** Activated when keywords associated with scheduling are detected in the query (e.g., day, time slot, timetable). Retrieves top-3 embeddings from **time_table** and optionally augments with a MySQL lookup for precise room and time data.
- **Curriculum:** Activated when course-related keywords are present (e.g., subject name, credit, course code). Performs semantic search over the **curriculum** collection (top-3 chunks) to retrieve programme-specific information.

- **University Information:** Default fallback channel. Retrieves from **uni_info** when no other channel matches, covering campus layout, building zones, and general facilities within Building E-12 at KMITL.
- **General Chat:** Handles open-ended conversational queries without external retrieval. The LLM generates responses solely from its parametric knowledge and the current session history.

Figure 3.3 illustrates how the keyword classifier routes each incoming query through the appropriate retrieval channel before LLM generation.

Figure 3.3 RAG Query Routing: Keyword Classifier Directing Queries to Six Retrieval Channels

Collection	Size	Content	Query Signal
curriculum	516 chunks	Course descriptions, learning outcomes, prerequisites, and credit structures for Years 1–4 (RAI programme)	Course name and subject code keywords
time_table	128 entries	Structured Thai sentences encoding (day, time slot, course name, room) tuples	Schedule-related keywords
uni_info	7 entries	Campus buildings, facilities, and zones within Building E-12 at KMITL	Building and zone queries
conversation_memory	Dynamic	LLM-summarised past session transcripts, tagged per **student_id**	Semantic similarity + student_id filter

Table 3.3 Milvus Vector Database Collections and Retrieval Signals

3.4.3.2 Personalised Prompt Generation

Responses are dynamically tailored using student metadata, including name and academic year. Tone adaptation is applied based on student level, ensuring appropriate communication style. Prompts also include recent conversation summaries and retrieved contextual knowledge to maintain coherence.

The system prompt specifies the robot's persona as a female Thai-language assistant with the following behavioural constraints:

- All speech particles must be feminine; masculine particles and first-person masculine pronouns are prohibited.
- The student is addressed by nickname only; full names are never used mid-conversation.
- Responses are capped at two sentences to ensure TTS output remains concise and intelligible.
- Numbers, acronyms, and building codes are expanded into their full Thai phonetic representations to ensure correct TTS pronunciation (e.g., E-12 is rendered as its Thai phonetic equivalent; room numbers are spoken digit-by-digit).

3.4.3.3 Output Schema and Intent Classification

The LLM returns structured JSON outputs containing response text and intent classification. Supported intents include:

- **chat / info:** Triggers speech output only.
- **navigate:** Triggers speech and ROS2 navigation commands.
- **farewell:** Ends session and resumes autonomous roaming.

3.4.3.4 Session Memory and Persistence

The system uses a dual storage strategy:

- **SQLite:** Stores full conversation logs for auditing and debugging.
- **Milvus:** Stores summarised embeddings for long-term semantic memory.

Session states are maintained in-memory with timeout policies (30 minutes for registered users, 5 minutes for guests). A silence-based timeout mechanism triggers reprompting and automatic session termination. A per-student greeting cooldown of 300 seconds is enforced by the **/greeting** endpoint to prevent repeated activation caused by transient face detection events within a single encounter.

3.4.4 Speech-to-Text and Grammar Correction

Speech processing is divided into two stages: transcription and linguistic refinement.

3.4.4.1 ASR via Typhoon2-Audio

Audio input is sent to the Typhoon2-Audio sidecar via a POST **/stt** request. The model processes 16 kHz PCM audio and returns Thai transcription with typical latency of 1–3 seconds.

The system supports two operational STT modes. Mode B is the recommended production configuration as it leverages GPU-accelerated inference on the server and eliminates local model loading on the Pi.

- **Mode A (edge STT):** Pi 5 #1 performs on-device transcription and sends the resulting text as a JSON payload to the server via **/detection**. Two backends are available under Mode A. The recommended backend is **scb10x/typhoon-asr-realtime** via the NVIDIA NeMo Toolkit (**–stt-backend typhoon-local**), achieving

0.68–2.0 seconds per segment on Pi 5 CPU with no hallucinations observed in live testing. The fallback backend is **biodatlab/distill-whisper-th-small** via faster-whisper INT8 quantisation (**–stt-backend whisper**), achieving 8.5–9.7 seconds per segment and requiring hallucination mitigations including VAD filtering, average log-probability thresholding, and a **max_new_tokens** cap of 50.

- **Mode B (server STT):** Pi 5 #1 forwards raw PCM WAV audio as a multipart payload to **/audio_detection**. The server invokes the Typhoon2-Audio-8B sidecar for GPU-accelerated transcription before the remaining inference pipeline proceeds. This mode requires no local STT model on the Pi, enabling startup in approximately 3 seconds.

Backend	Speed (Pi 5 CPU)	RAM	Role
scb10x/typhoon-asr-realtime (NeMo)	0.68–2.0 s/seg	~2.8 GB + 1.7 GB swap	Mode A primary
biodatlab/distill-whisper-th-small (faster-whisper INT8)	8.5–9.7 s/seg	~800 MB–1 GB	Mode A fallback
Typhoon2-Audio-8B (server GPU)	1–5 s/seg	GPU (server)	Mode B (recommended)

Table 3.4 STT Backend Comparison for Edge Audio Coordination Subsystem

3.4.4.2 Grammar Normalisation

The 8B LLM corrects spelling and phonetic errors while preserving meaning. Strict constraints ensure that no additional content is introduced. Grammar correction is bypassed when the STT confidence score exceeds a threshold of 0.85, avoiding unnecessary LLM invocation for high-confidence transcriptions.

3.4.4.3 Robustness Mechanisms

Several safeguards are implemented:

- Minimum input length filtering – transcriptions shorter than three characters are rejected by a noise gate and do not trigger a server inference call.
- Output length validation (50–150%) – the grammar-corrected output must fall within 50–150% of the input character length. Outputs outside this range trigger reversion to the original raw transcription.
- Timeout fallback to raw transcription – if the grammar correction LLM call exceeds a configurable timeout, the raw transcription is used without correction to maintain conversational responsiveness.

3.4.5 Text-to-Speech Module

The TTS subsystem converts responses into spoken Thai using a preprocessing and synthesis pipeline. It handles Thai phonetic structures and converts non-Thai elements into readable forms.

Text is sent to the Typhoon2-Audio sidecar via **/tts**, where it is synthesised into 16 kHz PCM WAV audio. The output is then forwarded to Raspberry Pi 5 for playback. The sidecar is preferred due to its faster GPU-based inference compared to edge-only processing.

3.4.5.1 Text Normalisation

Preprocessing includes:

- English and acronym expansion.
- Number-to-Thai conversion.
- Unicode normalisation.
- Word and syllable segmentation.

3.4.5.2 Asynchronous Dispatch

TTS and navigation commands are executed asynchronously using **asyncio**, ensuring the main pipeline remains non-blocking and responsive. This pattern eliminates the 15–25 second blocking observed when TTS was awaited synchronously on slow Raspberry Pi 5 networks, freeing the server to accept the next student interaction immediately.

3.4.6 Audio Sidecar Service

The Audio Sidecar is an independent FastAPI process running on the AI Brain server on port 8001. It is isolated in a separate Python 3.10 virtual environment to avoid model dependency conflicts with **capstone_server**. The service is backed by the **typhoon-ai/llama3.1-typhoon2-audio-8b-instruct** multimodal model, loaded once at server startup in **float16** precision with **attn_implementation="eager"**. Flash-attention is intentionally disabled because **GLIBC_2.32** is unavailable on the Ubuntu 20.04 host environment. The model is shared across all requests via a singleton pattern to eliminate repeated load overhead.

Figure 3.4 illustrates the two independent processing flows—TTS synthesis and STT transcription—exposed through the sidecar's endpoints.

Figure 3.4 Audio Sidecar Service: TTS Synthesis and STT Transcription Processing Flows (Port 8001)

3.4.6.1 TTS Synthesis Flow

The sidecar receives a Thai text string from **capstone_server** via **POST /tts {"text": "..."}**. It invokes **model.synthesize_speech(text)**, which returns a **float32** NumPy audio array at a 16 kHz sampling rate. The array is encoded to a BytesIO buffer using **soundfile** in WAV format with PCM_16 subtype, and returned as raw WAV bytes with **Content-Type: audio/wav**. Typical output is approximately 105 KB per utterance with a synthesis latency of approximately 633 ms on GPU.

3.4.6.2 STT Transcription Flow

The sidecar receives raw WAV bytes in the HTTP request body via **POST /stt**. Because the model API requires a file path rather than an in-memory buffer, the bytes are first written to a temporary file. A multi-turn conversation is then constructed with a system instruction and a user message containing an **audio** content block pointing to the temporary file. The model generates a transcription with parameters **max_new_tokens=256**, **do_sample=False**, and **temperature=1.0**. The temporary file is deleted in a **finally** block regardless of success or failure. Typical transcription latency is 1–5 seconds on the GPU server.

Method	Path	Input	Output
POST	/tts	{"text": str}	WAV bytes (audio/wav)
POST	/stt	Raw WAV bytes body	{"text": str}
GET	/health	—	{"status": "ok", "model": model_id}

Table 3.5 Audio Sidecar API Endpoints (Port 8001)

3.4.7 Inference Pipeline

Each request to **/detection** (text payload from edge STT) or **/audio_detection** (raw WAV with metadata from Mode B) triggers a sequential eight-stage pipeline on the AI Brain server. Figure 3.5 provides a stage-level overview of the complete flow from input filtering to asynchronous response dispatch.

Figure 3.5 AI Inference Pipeline: Eight Sequential Processing Stages from Input Filter to Asynchronous Dispatch

1. **Input Filter (Noise Gate).** Transcriptions shorter than three characters are rejected without triggering any downstream model inference. This eliminates spurious activations from coughs, brief sounds, or single-character STT outputs caused by background noise.

2. **Grammar Correction.** The lightweight Typhoon2-8B model corrects spelling and phonetic errors introduced by ASR (e.g., mishearing Thai tone marks or vowel clusters). Correction is bypassed entirely when the STT confidence score exceeds 0.85, avoiding an unnecessary LLM call for high-quality transcriptions. A length guard ensures the corrected output falls within 50–150% of the input character length; outputs outside this range revert to the original raw transcription. A timeout fallback also reverts to raw transcription if the correction call exceeds the configured limit.

3. **Query Routing and Context Retrieval.** A keyword-based classifier maps the query to one of six retrieval channels (**conversation_memory**, **student_info**, **timetable**, **curriculum**, **uni_info**, or **general_chat**), determining which Milvus collection or MySQL table is queried in the next stage. The top-3 most semantically relevant chunks are retrieved using BAAI/bge-m3 embeddings and cosine similarity search.

4. **Memory Retrieval.** For registered students, the top-3 most semantically similar past conversation summaries are retrieved from the **conversation_memory** Milvus collection, filtered by **student_id**. A MySQL query also retrieves the student's academic year and nickname for personalised prompt tone and address style.

5. **LLM Response Generation.** The Typhoon2-70B model (Q5_K_M GGUF via Ollama) generates a Thai-language response. The system prompt enforces the robot's persona: feminine speech particles, nickname-only address, a maximum of two sentences, and phonetic expansion of building codes and numbers for TTS clarity (e.g., building code E-12 is expanded to its Thai phonetic equivalent; room numbers are spoken digit by digit). The model also classifies the response intent in structured JSON output.

6. **Asynchronous Logging and Memory Update.** Session history is updated after each exchange. An end-of-session memory summarisation task is enqueued asynchronously so that memory writes do not block

the main response pipeline.

7. **Intent-based Dispatch (TTS / Navigation).** The response text undergoes preprocessing—acronym expansion, number-to-Thai conversion, Unicode normalisation, and word segmentation—before being sent to the Audio Sidecar via **POST /tts**. All downstream commands (WAV delivery to Pi 5 #1 at **/audio_play**, microphone activation flag at **/set_active**, and navigation commands to Pi 5 #2 at **/nav_state** when intent is **navigate**) are fired concurrently using **asyncio**, eliminating the 15–25 second serial delay observed when these calls were awaited synchronously.

8. **System Reactivation.** The microphone activation flag (**/set_active active=1**) is sent after audio playback completes, not immediately, ensuring the robot cannot transcribe its own synthesised speech output.

Stage	Model / Component	Typical Latency
Grammar correction	Typhoon2-8B	800–1,500 ms
RAG retrieval	Milvus + bge-m3	50–200 ms
Memory retrieval	Milvus + MySQL	30–80 ms
LLM generation	Typhoon2-70B	4,000–12,000 ms
TTS synthesis	Typhoon2-Audio-8B	~633 ms
Network (Pi 5 transfer)	WAV delivery	~50 ms
Total end-to-end	—	**6–15 s per turn**

Table 3.6 AI Inference Pipeline Latency Summary (Single GPU Server)

3.4.8 Vision and Face Recognition Subsystem

The vision subsystem (JetsonVision) is the first processing stage in the interaction pipeline. It operates continuously on the Jetson Orin Nano and is responsible for detecting, tracking, and identifying human faces in the robot's camera field of view. The processing stages—from raw camera frame to WebSocket event dispatch—are illustrated in Figure 3.6.

Figure 3.6 JetsonVision Pipeline: Face Detection, Multi-Object Tracking, Head Pose Gating, and Recognition

3.4.8.1 Detection and Tracking

Face detection is performed using the SCRFD (Sample and Computation Redistribution Face Detector) model with a 640×640 input resolution. The detector is configured with a confidence threshold of 0.5 and a Non-Maximum Suppression (NMS) IoU threshold of 0.4. The model is accelerated using TensorRT via the ONNX Runtime TensorRT execution provider when operating in GPU mode; a CPU-only ONNX fallback is available via the **–cpu-only** command-line flag.

SCRFD produces bounding boxes and five facial landmark points per detected face. These outputs are passed to a ByteTracker multi-object tracker, which assigns a persistent **track_id** to each face across consecutive frames. ByteTracker maintains track continuity through brief occlusions and frames in which detection confidence drops below the threshold, providing stable identity anchors for downstream recognition.

3.4.8.2 Head Pose Estimation and Frontal Gate

A lightweight head pose estimator computes the yaw, pitch, and roll of each tracked face from the five SCRFD landmark points. Only faces satisfying the frontal constraint of yaw less than 25 degrees and pitch less than 15 degrees are eligible for recognition. This gate prevents recognition attempts on profile views, where ArcFace embedding quality degrades significantly, thereby reducing false-positive matches.

3.4.8.3 Face Recognition

Eligible face crops are passed to an ArcFace model with a ResNet backbone that produces 512-dimensional face embeddings. Inference is performed in FP16 precision to utilise the Tensor Cores available on the Jetson Orin Nano's GPU. A cooldown gate of 5 seconds per **track_id** ensures that repeated recognition calls for the same continuously-present face do not saturate the system.

Recognition is performed by computing the cosine similarity between the query embedding and each stored embedding in **data/enrollments.json**. If the maximum similarity across all enrolment entries for a given identity exceeds a configurable threshold, the face is labelled as a known student and the associated metadata is returned; otherwise, the face is labelled unknown and assigned a temporary UUID as its **person_id**.

Student enrolment is performed offline using the **enroll_student.py** tool, which guides the operator to capture five face angles (frontal, left, right, and two 45-degree intermediate poses) and stores the corresponding embeddings alongside the student's Thai and English full names and nickname.

3.4.8.4 WebSocket Event Emission

Upon a successful recognition or unknown-face detection event, the Jetson transmits a JSON message to Pi 5 #1 over the persistent WebSocket connection on port 8765. The message schema carries a top-level **status** field set to **detected**, and a nested **metadata** object containing the fields **eng_name**, **thai_name**, **student_id**, **is_registered**, and **confidence**. For unrecognised faces, **is_registered** is set to false and the name fields are null.

On the receiving side, the Pi 5 VisionReceiver applies a confidence threshold of 0.75 before forwarding detection events to the coordinator. A 15-second debounce is applied per face, ensuring that the detection callback fires at most once every 15 seconds for a continuously present individual, preventing the asyncio queue from being flooded during the STT processing window. Null identity fields are mapped to the string value

Unknown before use in downstream payloads.

The WebSocket message emitted on each detection or recognition event carries the following JSON structure:

```
{
  "status": "detected",
  "metadata": {
    "eng_name": "Somchai Jaidee",
    "thai_name": "      ",
    "student_id": "66011386",
    "is_registered": true,
    "confidence": 0.87
  }
}
```

For unrecognised faces, **is_registered** is set to **false**, the name fields are **null**, and a temporary UUID is assigned as the **person_id** in the downstream payload.

3.4.9 Edge Audio Coordination Subsystem

The edge audio coordination subsystem (RaspiReceive) deployed on Pi 5 #1 serves as the interaction hub. It bridges the vision subsystem and the AI server by combining face detection events with speech transcription results into a unified payload, while managing audio capture and playback. The complete audio coordination flow—from microphone capture through VAD, STT, combiner gating, and HTTP dispatch—is shown in Figure 3.7.

Figure 3.7 RaspiReceive Pipeline: Pre-emphasis, Voice Activity Detection, STT, Combiner Gating, and Server Dispatch

3.4.9.1 Concurrent Task Architecture

RaspiReceive is implemented as a single **asyncio** event loop running four concurrent coroutines via **asyncio.gather**:

- **_detection_loop():** Listens on WebSocket port 8765 for incoming Jetson events, applies session upgrade logic from Unknown to Known, and enqueues events for the combiner.
- **_stt_consumer_loop():** Drains the output queue of the AudioWorker background thread, accumulating transcription results in a rolling recent buffer.
- **_combiner_loop():** Executes every 500 ms, matching detection events with transcription results according to the gating rules described below, and dispatching combined payloads to the AI server.
- **_run_activation_server():** Hosts the FastAPI HTTP service on port 8766, providing endpoints for audio playback, microphone activation control, and health monitoring.

A background AudioWorker thread, running outside the asyncio event loop, handles blocking microphone input/output via PyAudio at 16 kHz mono. Captured audio frames are accumulated in a three-second rolling circular buffer from which speech segments are extracted by the SileroVAD model. The AudioWorker is paused during TTS playback and resumed upon completion, preventing the robot from transcribing its own synthesised speech output.

The activation state is managed through a push-based model: the server posts **/set_active** with **active = 1** to re-enable the microphone after each response, and **active = 0** to suppress new input during processing. An auto-reactivation timer of 60 seconds serves as a fallback in the event that the server call does not arrive.

3.4.9.2 Voice Activity Detection and Transcription

A two-tier VAD pipeline is employed to eliminate hallucinations caused by silent or low-energy ambient audio. In the first tier, a pre-emphasis filter boosts the amplitude of Thai tonal consonants before analysis. In the second tier, the SileroVAD model assigns a speech probability to each audio chunk; chunks with probability exceeding the threshold of 0.73 are classified as speech. Only frames passing both tiers are appended to the utterance accumulator. This two-tier approach eliminates the hallucination loops observed with single-stage VAD, where models such as faster-whisper would generate repeated Thai or English phrases on background noise.

Upon detecting the end of a speech segment, the accumulated audio is submitted to the active STT backend. The transcription result is wrapped in an **STTChunk** dataclass and placed on the output queue consumed by **_stt_consumer_loop()**. The microphone input is automatically muted during TTS audio playback by pausing the AudioWorker, preventing the robot from transcribing its own synthesised speech. The AudioWorker resumes microphone capture only after playback has completed.

3.4.9.3 Detection-Speech Combiner Logic

The **_combiner_loop()** implements four sequential gating rules before dispatching a payload to the AI server:

1. **Consecutive detection gate:** At least two detection events for the same **track_id** must arrive within a ten-second sliding window before a send is triggered. This prevents false activations from brief, transient face detections.

2. **Unknown-to-Known upgrade window:** When an unknown face is first detected, the combiner holds the event for up to ten seconds (**unknown_hold_sec = 10**) waiting for the same **track_id** to receive a known identity from a subsequent Jetson recognition event.

3. **STT wait:** After a valid detection event passes the consecutive gate, the combiner waits up to 60 seconds (**stt_wait_sec = 60**) for a non-empty transcription result. If the timeout expires, the payload is dispatched with an empty text field, triggering a contextual greeting on the server side.

4. **Post-send cooldown:** After dispatching a payload, the combiner ignores new detection events for five seconds (**cooldown_sec = 5**) to prevent duplicate triggers within the same interaction.

Parameter	Default	Description
websocket_port	8765	Jetson detection listener port
audio_vad_threshold	0.73	SileroVAD speech probability gate
stt_model_name	distill-whisper-th-small	Whisper model ID
cooldown_sec	5.0	Post-send detection cooldown (s)
unknown_hold_sec	10.0	Unknown-to-Known upgrade window (s)
stt_wait_sec	60.0	Maximum wait for STT result (s)
consecutive_detections_required	2	Detections needed within window
activate_timeout_sec	60.0	Auto-reactivate if server silent (s)
max_tts_duration_sec	30.0	Playback watchdog timeout (s)

Table 3.7 RaspiReceive Subsystem Configuration Parameters

3.4.9.4 Audio Playback and Watchdog

Synthesised audio (WAV) received at the **/audio_play** endpoint is queued for sequential playback via the system's **aplay** command. A background watchdog task monitors each aplay subprocess and force-terminates it if playback has not completed within 30 seconds (**max_tts_duration_sec = 30**). This safeguard prevents the audio pipeline from entering a blocked state due to network transfer errors or corrupted WAV data.

3.4.10 Expressive Robot Interface Subsystem

The expressive interface subsystem (baymax-face-ui) renders an animated robot face on the HDMI display connected to Pi 5 #1. The system communicates the robot's internal state to the interacting student through five discrete emotional expressions that are mapped to interaction phases.

3.4.10.1 Expression System

The face is rendered on an HTML5 Canvas element within a fullscreen Electron.js BrowserWindow. Animation is driven by a requestAnimationFrame loop with interpolated transitions between expression states. Table 3.4 describes the five supported expressions, and Figure 3.8 illustrates the complete state transition sequence across a full interaction cycle, from person detection through response playback and navigation.

Figure 3.8 Robot Face Expression Flow: State Transitions Driven by Interaction Lifecycle Events

Code	Name	Visual Description	Triggered By
0	Idle	Neutral with slow breathing animation	Playback ends (chat / info / farewell)
1	Scanning	Eyes sweep left-right	Navigating to a destination
2	Happy	Wide smile, sparkle particles	Person detected, greeting about to fire
3	Talking	Mouth animates vowel shapes	TTS audio playback in progress
4	Thinking	Drooped eyelid, drifting iris	STT ready, awaiting LLM response

Table 3.8 Robot Face Expression States

3.4.10.2 IPC Architecture

All face emotion transitions are managed locally by Pi 5 #1. The server does not call the face service directly. Instead, the **raspi_main.py** coordinator drives all transitions based on its own visibility into the interaction lifecycle: it sets **happy (2)** as soon as a person is identified and before posting to **/greeting**; it sets **thinking (4)** when the STT result is ready and the POST to the server is about to be dispatched; it sets **talking (3)** when audio playback begins; and it sets **idle (0)** when playback ends. For navigation intents, the transition after playback ends is **scanning (1)** rather than **idle (0)**, persisting until the next interaction.

Face calls are issued as fire-and-forget HTTP requests so that a slow or unresponsive face service never blocks the audio delivery pipeline. A Python FastAPI service (**pi5_face_service.py**) listens on port 7000 and accepts HTTP POST requests at the **/face_emotion** endpoint. The request body specifies the target emotion code as a JSON integer field. The FastAPI process forwards emotion codes to the Electron renderer process via an internal WebSocket channel on **localhost:8768**, which acts as an IPC bridge. The Electron main process listens on this channel and invokes the appropriate canvas state transition in the renderer. This two-process architecture decouples the HTTP-facing service from the rendering engine, enabling independent restart of either component without disrupting the other.

3.4.11 Autonomous Navigation Subsystem

The navigation subsystem (Ros2_robot) deployed on Pi 5 #2 provides autonomous mobile navigation within the E12 Building floor at KMITL. It integrates hardware drivers, a LiDAR-based localisation stack, and a path planning framework, exposed to external control via an HTTP bridge.

3.4.11.1 Hardware Layer and Odometry

The robot chassis is modelled in URDF via the **ired.urdf.xacro** file, which defines the **base_link**, **base_footprint**, left and right drive wheel joints, IMU frame, and RPLiDAR mounting point. The iREDCr differential drive controller communicates with the wheel motors over a serial interface at **/dev/ttyiREDCr** and publishes wheel odometry to the **/odom_encoder** topic.

A supplementary odometry source is provided by a scan-to-scan laser scan matcher (ros2_laser_scan_matcher), which publishes its estimate to the **/odom** topic. The scan matcher is configured with keyframe thresholds of 0.05 m and 0.087 rad (approximately 5 degrees), triggering a new scan match only when the robot has moved or rotated beyond these limits. This laser-derived odometry exhibits lower drift under wheel slippage conditions and is used as the primary input to the AMCL particle filter.

3.4.11.2 Localisation and Path Planning

Localisation is performed by Adaptive Monte Carlo Localisation (AMCL) against a pre-built 2D occupancy grid map of the operating environment. The particle filter is seeded from an initial pose estimate; when no initial pose is available at startup, the robot executes a 360-degree in-place rotation to gather sufficient scan data for the filter to converge.

Path planning and execution are managed by the Nav2 stack. The global planner generates a collision-free path from the current estimated pose to the goal pose, and the DWB (Dynamic Window B) local controller tracks this path. The DWB controller is configured with a maximum translational velocity of 0.3 m/s and a stopped-velocity threshold of 0.25 m/s. The stopped-velocity threshold must be set strictly less than the maximum velocity to prevent the controller from interpreting normal cruising speed as a stopped condition, which was found during development testing to cause an indefinite in-place spinning behaviour.

3.4.11.3 HTTP Bridge and Navigation State Machine

An HTTP bridge service (**pi5_service.py**), deployed as a standalone Python FastAPI application on port 8767, translates external HTTP commands from the AI Brain server into ROS2 navigation goals. The bridge exposes a **/nav_state** endpoint that accepts a JSON body containing the fields **state** (integer 0, 1, or 2) and **destination** (room name string). The full navigation state machine, including goal dispatch, stuck-detection sub-sequence, and retry logic, is illustrated in Figure 3.9.

Figure 3.9 Autonomous Navigation State Machine: Goal Dispatch, Stuck Detection (360° Recovery), and Retry Logic

State	Label	Action
0	Stop	Cancel active navigation goal; robot halts in place
1	Roam	Cycle through patrol waypoints continuously
2	Navigate	Execute goal-directed navigation to a named room

Table 3.9 Navigation State Definitions

Destination room names are resolved to metric goal poses (x, y, yaw in the map frame) via a static mapping defined in **goals.yaml**. The room-to-codename mapping is as follows: **e12-1** maps to label A, **e12-2** maps to label B, and **e12-3** maps to label C. Goal poses are determined experimentally by driving the robot to each target location and recording the AMCL pose estimate from the **/amcl_pose** topic.

The **navigation_manager.py** ROS2 node wraps the Nav2 BasicNavigator API and publishes the current navigation status to the **/navigation_status** topic with values IDLE, ROAMING, NAVIGATING, SUCCEEDED, or FAILED. The node implements a stuck-detection mechanism: if the robot's displacement over a 15-second window is less than 0.05 m, it is considered stuck, and the node commands a 360-degree in-place rotation to help AMCL re-localise before retrying the goal. A maximum of two retries is permitted; if both fail, the status is published as FAILED and the state machine returns to IDLE.

3.4.12 Inter-subsystem Communication Architecture

The protocol selection for each communication link is motivated by the traffic characteristics of that link.

Link	Protocol	Rationale
Jetson to Pi 5 #1	WebSocket (persistent)	Continuous event stream; avoids TCP handshake overhead
Pi 5 #1 to Server	HTTP POST	Request-response matches trigger-and-wait pattern
Server to Pi 5 #1 audio	HTTP POST	One-shot delivery of binary WAV payload
Server to Pi 5 #1 face	HTTP POST	Fire-and-forget display state command
Server to Pi 5 #2	HTTP POST	Decouples AI server from ROS2 middleware
Pi 5 #2 bridge to Nav	ROS2 topic (std_msgs/String)	Standard ROS2 IPC within single device

Table 3.10 Communication Links and Protocol Selection Rationale

3.4.13 System Data Flow and Interaction Scenarios

3.4.13.1 Scenario A: Registered Student – Conversational Query

1. The Jetson detects a frontal face and extracts a 512-dimensional ArcFace embedding. Cosine similarity search over the enrolment database yields a match, and a WebSocket detection event is transmitted to Pi 5 #1 with **is_known** set to true.

2. The student speaks; SileroVAD detects voice activity above the 0.73 threshold and the Whisper model produces the transcription.

3. The combiner confirms two detection events within the ten-second window, assembles the combined payload, and dispatches it via POST to **/detection**.

4. The server applies grammar correction (bypassed if confidence exceeds 0.85), classifies the query intent, and routes to the appropriate RAG channel.

5. Milvus returns the top-3 semantically similar knowledge entries. The 70B LLM generates a two-sentence Thai response and classifies the intent as **chat** or **info**.

6. The Typhoon2-Audio sidecar synthesises the response into a WAV file (approximately 105 KB, approximately 633 ms synthesis time).

7. The server concurrently dispatches: (a) WAV to **/audio_play**, (b) active = 0 to **/set_active** to mute the microphone during playback.

8. Pi 5 #1 receives the WAV, sets emotion code 3 (Talking) on the local face display, and plays the audio through the speaker via **aplay**.

9. Upon playback completion, Pi 5 #1 sets emotion code 0 (Idle). The server then sends active = 1 to **/set_active**, re-enabling the microphone for the next utterance.

3.4.13.2 Scenario B: Navigation Request

Steps 1 through 5 are identical to Scenario A, with the LLM classifying the intent as **navigate** and identifying the destination room.

6. The server dispatches: (a) synthesised WAV of the verbal confirmation to **/audio_play**, (b) navigation command with state = 2 and the destination room name to **/nav_state** on Pi 5 #2. Pi 5 #1 sets emotion 3 (Talking) when playback begins, then emotion 1 (Scanning) when playback ends, reflecting the robot's transition into navigation mode.

7. Pi 5 #2 receives the command, resolves the room name to the corresponding goal pose, and invokes BasicNavigator.goToPose().

8. Nav2 plans a path using the AMCL-estimated current pose and the DWB controller executes trajectory following at up to 0.3 m/s.

9. Upon reaching the goal, navigation_manager publishes SUCCEEDED on **/navigation_status** and transitions the state machine to IDLE.

3.4.14 Session and State Management

3.4.14.1 Robot Interaction State

From the perspective of the RaspiReceive coordinator, the robot occupies one of two interaction states at any given time:

- **INACTIVE:** The microphone is disabled. The robot is either in standby or in playback mode. New detection events are queued but not acted upon.
- **ACTIVE:** The microphone is enabled and the combiner loop is actively matching detection events with speech input. The transition from INACTIVE to ACTIVE occurs when the server sends **set_active = 1**, either upon TTS playback completion or after the **activate_timeout_sec** (60 s) timer fires.

3.4.14.2 Student Session Lifecycle

On the AI Brain server, each interaction is associated with a session object identified by the **person_id** field of the incoming payload. The session lifecycle proceeds as follows:

1. **Session creation:** A new session is created on the first detection event for a given **person_id**. The session stores the student's metadata, initialises an empty short-term history buffer, and loads the top-3 relevant long-term memory summaries from Milvus.
2. **Session continuation:** Subsequent detection events within the timeout window extend the session. Short-term history is updated after each exchange.
3. **Session expiry and persistence:** If no detection event is received within the timeout period (30 minutes for registered students, 5 minutes for guests), the session is closed. For registered students, the 8B LLM summarises the session transcript and the resulting embedding is stored in the **conversation_memory** Milvus collection.

3.4.14.3 Navigation State Machine

The navigation state machine on Pi 5 #2 transitions between the following states:

- **IDLE:** No active goal. The robot is stationary and awaiting the next command from the HTTP bridge.
- **ROAMING:** The robot is cycling through pre-defined patrol waypoints. Transitions to IDLE on receipt of state = 0.
- **NAVIGATING:** The robot is actively pursuing a specific goal pose. Transitions to SUCCEEDED on goal completion, to IDLE on state = 0, or enters the stuck-recovery sub-sequence if no progress is detected for 15 seconds.

- **SUCCEEDED:** The goal has been reached. Automatically transitions back to IDLE after publishing the status.
- **FAILED:** Two consecutive navigation attempts have failed. The state machine resets to IDLE and logs the failure for monitoring.

CHAPTER 4

REFERENCE AND CITATION

4.1 Development Environment and Dependencies

The AI Brain server runs on Ubuntu 22.04 with Python 3.10 in a virtual environment. Core dependencies include FastAPI (v0.124.4) for the HTTP framework, Uvicorn as the ASGI server, PyMilvus (v2.6.10) for vector database access, PyThaiNLP (v3.1.1) for Thai text processing, PyThaiTTS (v0.4.2) for text normalisation preprocessing, PyTorch (v2.4.1) for GPU-accelerated tensor operations, and **mysql-connector-python** for MySQL connectivity. Docker Compose is used to orchestrate the Milvus cluster and MySQL service. Model management and inference are handled by Ollama, which serves the **llama3.1-typhoon2-70b-instruct** model (Q5_K_M GGUF quantisation) for main response generation and **llama3.1-typhoon2-8b-instruct** for grammar correction and session summarisation.

The Audio Sidecar service (**audio_package**) runs in an isolated Python 3.10 virtual environment on the same AI Brain server at port 8001 to prevent model dependency conflicts with the main inference service. It is backed by the **typhoon-ai/llama3.1-typhoon2-audio-8b-instruct** multimodal model, loaded once at server startup in **float16** precision with **attn_implementation="eager"** to maintain compatibility with the Ubuntu 20.04 host environment. This service handles both text-to-speech synthesis (POST **/tts**) and speech-to-text transcription (POST **/stt**), producing 16 kHz PCM_16 WAV audio output and Thai-language text transcriptions respectively. A singleton model pattern eliminates repeated model load overhead across concurrent requests.

4.2 Database Setup and Knowledge Ingestion

The Milvus vector database consists of four collections, each indexed using the **IVF_FLAT** method with **nlist** set to 128 and queried with **nprobe** set to 10. All collections use **BAAI/bge-m3** to produce 1024-dimensional dense embeddings and apply cosine similarity for approximate nearest-neighbour search. The **conversation_memory** collection stores per-session LLM-generated summaries, tagged by **student_id** to enable filtered semantic retrieval of long-term interaction history.

The three RAG collections — **curriculum**, **time_table**, and **uni_info** — are populated using ingestion tools located in the **tools/** directory. The **curriculum** collection is derived from the RAI programme specification document, producing 516 Thai-language chunk embeddings covering course descriptions, learning outcomes, prerequisites, and credit structures for Years 1 through 4.

The **time_table** collection is re-ingested from an academic Excel dataset and transformed into structured Thai sentences per (day, time slot, course name, room) tuple to maximise retrieval precision, resulting in 128 entries. The **uni_info** collection contains 7 entries describing campus buildings, facilities, zones, and departmental information within Building E-12 at KMITL.

The MySQL relational database maintains five tables: **Academic_Year** (year boundary and tone adaptation metadata), **Students** (student identity, full name and nickname in Thai and English, academic year, and enrolment status), **Face_Recognition_Data** (biometric face encodings linked to student identities), **ExcelTimetable-**

Data (structured timetable records for precise SQL-based schedule lookups), and a logging table for system monitoring. Timetable data is synchronised across both storage systems: MySQL stores structured records for exact lookups while Milvus stores corresponding embeddings for semantic retrieval.

4.3 API Endpoint Behaviour

The **/greeting** endpoint is triggered once at the start of each session. It is rate-limited using a 300-second cooldown per **person_id** to prevent repeated activation caused by transient face detection events. The greeting pipeline retrieves the student's academic year from MySQL, fetches the top-3 semantically similar memory summaries from the **conversation_memory** Milvus collection filtered by **student_id**, and invokes the Typhoon2-70B LLM to generate a personalised Thai greeting. Concurrently, a ROS2 **stop_roaming** command is dispatched to Pi 5 #2 to halt autonomous navigation. The session history is initialised with this greeting as a bot-only entry, ensuring that the first **/detection** call has prior conversational context.

The **/detection** endpoint (text payload from edge STT) and **/audio_detection** endpoint (raw WAV audio from Mode B server STT) both execute the full eight-stage inference pipeline described in Chapter 3. A noise gate filters out transcriptions shorter than three characters before any model inference is triggered. Registered and guest users follow separate execution paths: guest users bypass MySQL queries, Milvus memory retrieval, and end-of-session memory storage, but still receive full RAG-based chatbot responses for general queries. Upon pipeline completion, the system asynchronously dispatches the synthesised WAV audio to the Raspberry Pi 5 **/audio_play** endpoint and sends **active=1** to the **/set_active** endpoint to re-enable the microphone for the next interaction. Navigation commands are conditionally sent to Pi 5 #2 **/nav_state** when the LLM classifies the response intent as **navigate**.

4.4 Performance Evaluation

System performance was evaluated across two test rounds. Test 1 assessed end-to-end pipeline accuracy using 30 curated Thai-language queries covering all five RAG retrieval routes (timetable, curriculum, university information, student information, and general chat). Test 2 evaluated grammar correction performance on 20 simulated ASR error transcriptions representing common phonetic misrecognition patterns in Thai.

Stage	Component	Typical Latency	Notes
Grammar correction	Typhoon2-8B (Ollama)	800–1,500 ms	temp=0.1, max 256 tokens
RAG retrieval	Milvus + bge-m3	50–200 ms	IVF_FLAT, top-k=3
Memory retrieval	Milvus + MySQL	30–80 ms	filtered by **student_id**
LLM inference	Typhoon2-70B (Ollama)	4,000–12,000 ms	temp=0.7, max 512 tokens
TTS synthesis	Typhoon2-Audio-8B	~633 ms	GPU, ~3s utterance
Network (Pi 5)	WAV transfer	~50 ms	~105 KB typical
Total pipeline	—	6–15 s	End-to-end per turn

Table 4.1 Pipeline Timing Benchmarks (approximate, single GPU)

4.5 System Evaluation

Accuracy evaluation on the Test 1 benchmark shows that the RAG routing keyword classifier correctly identified the appropriate data source in 28 out of 30 queries (93.3%). The two misclassified cases involved ambiguous phrasing, which caused the system to fall back to the default **uni_info** route instead of the intended **curriculum** route. This fallback behaviour is by design: when no domain-specific keywords are detected, the system defaults to the **uni_info** collection to provide a safe and contextually relevant response rather than generating an empty reply.

LLM-generated responses were assessed by three independent evaluators, with 26 out of 30 responses (86.7%) rated as relevant and contextually appropriate. Intent classification achieved 100% accuracy across the evaluation set, correctly identifying all **chat**, **info**, **navigate**, and **farewell** intents from the structured JSON output produced by the Typhoon2-70B model.

In the Test 2 benchmark, grammar correction using the Typhoon2-8B model reduced the average word error rate (WER) from 18.4% in raw ASR output to 7.2% after correction, demonstrating a substantial reduction in transcription noise prior to RAG retrieval and LLM inference. In the few cases where correction degraded output quality (3 out of 20), the system's output length guard (50–150% of input character length) successfully detected the anomaly and reverted to the original raw transcription, preventing garbled or over-edited inputs from propagating through the inference pipeline.

Overall, the system demonstrates strong performance across all components. The combined accuracy and latency metrics indicate that the system is suitable for real-time interactive deployment within a university campus environment.

4.6 System Robustness

The system incorporates comprehensive error-handling mechanisms across all modules to ensure stable operation under both normal and fault conditions. In the event of LLM timeouts during main response generation, a predefined Thai fallback response is returned to the user to maintain conversational continuity without exposing internal errors. If Milvus vector retrieval fails at any stage, the system proceeds with empty context, allowing the Typhoon2-70B model to generate responses based solely on session conversation history and its parametric knowledge.

For MySQL query failures during student metadata retrieval, the **student_year** field defaults to 1 to prevent pipeline interruption and ensure personalised prompt construction can still proceed. TTS synthesis failures via the Audio Sidecar are logged for monitoring purposes but do not block downstream execution; the interaction pipeline continues and the microphone is re-enabled for the next utterance. Navigation commands to Pi 5 #2 are conditionally skipped when the configured navigation IP address contains **"TBD"**, allowing the system to operate in AI-only mode without the navigation module during development and testing.

On the edge side, the RaspiReceive audio coordinator implements an auto-reactivation timer of 60 seconds as a fallback in the event that the **/set_active** signal from the AI Brain server does not arrive, preventing the microphone from remaining permanently suppressed due to network interruptions. A 30-second watchdog timer monitors each **aplay** audio playback subprocess and force-terminates it if playback has not completed within the

configured duration, preventing the audio pipeline from entering a permanently blocked state.

All asynchronous fire-and-forget tasks are encapsulated within **try/except** blocks to prevent cascading failures, ensuring that errors in background processes such as memory summarisation and SQLite logging do not affect the main interaction pipeline.

4.7 Deployment and System Startup

A deterministic startup sequence must be followed to ensure all inter-node dependencies are satisfied before the system begins accepting interactions:

1. **AI Brain server:** Start Docker Compose services (Milvus, MySQL), launch the Ollama server with the Typhoon2-70B and Typhoon2-8B models loaded, launch the Typhoon2-Audio sidecar on port 8001, and start the FastAPI service on port 8000.
2. **Pi 5 #2 (navigation):** Launch the hardware bringup (**ros2 launch ired_bringup bringup.launch.py**), then the navigation stack (**ros2 launch ired_navigation navigation.launch.py**), then start the HTTP bridge (**python3 pi5_service.py**) on port 8767.
3. **Pi 5 #1 (audio + display):** Three user-level systemd services are managed via **systemctl –user**: **baymax-face-api** (FastAPI face service on port 7000), **baymax-face-ui** (Electron frontend), and **baymax-robot** (audio coordinator **raspi_main.py** on ports 8766 and 8765). Audio playback uses **aplay -D plughw:3,0**, where the ALSA device is addressed by card name (**-c Audio**) rather than by card number to ensure stable assignment across reboots. Mode B (server STT) is the recommended production configuration and requires no local STT model, enabling startup in approximately 3 seconds.
4. **Jetson Orin Nano:** Start the vision pipeline (**python3 visual_jetson_async.py –ws-enabled**). The **–ws-uri** defaults to **ws://10.26.9.196:8765**; the process connects to Pi 5 #1 and begins emitting detection events.

Graceful shutdown is coordinated as follows. The Jetson process handles SIGTERM, closes the WebSocket connection, releases the camera device, and exits cleanly. Pi 5 #1 handles SIGTERM by cancelling all asyncio tasks and flushing the audio playback queue before exit. The ROS2 navigation stack handles SIGTERM via standard lifecycle management, cancelling any active Nav2 goals prior to shutdown. Remote shutdown of the Jetson can be triggered from Pi 5 #1 using the **pi_shutdown_sender.py** utility, which transmits a TCP signal to the Jetson's shutdown listener on port 9999.

CHAPTER 5

BIBLIOGRAPHY AND REFERENCES

The writing of a bibliography or references should adhere to one system that aligns with the chosen citation method. Use 'Bibliography' when citing in-text with the name-year system and use 'References' when citing in-text with the numerical sequence system. The use of bibliography management software, such as Endnote, Mendeley, Zotero, etc., is permitted and encouraged.

5.1 GUIDELINES FOR WRITING BIBLIOGRAPHY

1. Print the word Bibliography in the center of the page in bold, size 24 points.
2. Arrange the entire reference list together, alphabetically based on the first letter of each reference, following the alphabetical order as per the dictionary.
3. For theses written in Thai, list the bibliography in the Thai language first, followed by the English language bibliography. For theses written in English, list the English language bibliography first, followed by the Thai language bibliography.
4. Begin printing the bibliography at the left edge of the paper. If the entry does not fit on one line, start a new line indented by 7-character spaces. Begin printing the 8th character if the entry extends to the second line. If not completed within two lines, continue onto lines 3-4 starting from the same position as line 2 until the end of the entry. When starting a new entry, align it with the left edge of the paper without skipping a line.

5.2 GUIDELINES FOR WRITING REFERENCES

1. Print the word References in the center of the page in bold, size 24 points.
2. Arrange the reference documents in numerical order, according to the numbers assigned within parentheses referring to the content of the thesis.
3. There is no need to separate by language or document type.
4. Print the number of each entry at the left edge of the paper. If the entry does not fit on one line, start a new line indented by 7-character spaces. Begin printing the 8th character if the entry extends to the second line. When starting a new entry, align it with the left edge of the paper without skipping a line.

5.3 METHOD OF LISTING BIBLIOGRAPHY

5.3.1 Authors

1. The author's name should not include any titles such as Mr., Mrs., Miss, Dr., etc.
2. For entries in the Thai language, use the author's first name followed by the last name.
3. For entries in English, use the last name, followed by a comma and the abbreviated first and middle names in sequence.
4. If the author holds a doctorate (Ph.D.), professorship, or other title, include it after the name, separated by a comma.

5. Use the name as it appears in the document for authors with honorific titles.

6. If there are two authors, use the name of the first author followed by and between the names of the two authors.

7. If there are more than two authors, list all authors.

8. For authors using pseudonyms, use the pseudonym as it appears in the document.

9. For translated documents, list the name of the translator first, followed by the phrase "translated from" and the original author's name. If the original title is unknown, use the phrase "translated by" or "translated and compiled by."

10. For documents attributed to an organization, ministry, or department, use the name of the organization as the author. In the case of documents issued under a specific department within a ministry or subordinate to a department, even if the ministry's name is mentioned, use the department's name as the author. Include the name of the subordinate unit in the publisher section.

11. For documents with only an editor or compiler, use the editor's name followed by a comma and the word editor or compiler after the name of the author.

12. For documents with no stated author or conference proceedings, use the title of the document as the first entry.

5.3.2 Article Titles

1. Use the title as it appears in the document, enclosed in quotation marks.

2. For English titles, capitalize the initial letter of each word, except for conjunctions, articles, and prepositions, unless they are the first word.

3. If there is a scientific name in Latin for plants, animals, etc., insert it in *italics* or <u>underline</u> it.

5.3.3 Book or Journal Titles

1. Print book or journal titles in bold.

2. For international journals, you may use the correct abbreviation of the journal name.

3. For book series, if referencing a specific volume, include only that volume. For example, if referring to volume 3, and if citing multiple non-consecutive volumes, indicate each volume number with commas in between, such as Volume 1, 3, 5. If citing all volumes in the series, state the total number of volumes, such as 5 volumes or 5 vol. Underline or print in bold (either one) continuously from the title.

5.3.4 Editions

1. For the first edition or first printing, it is not necessary to specify in the bibliography.

2. Specify the edition number for subsequent editions, as indicated in the printed material. For example, 2nd Ed., 2nd printing, 2nd, rev. ed., or 2nd, expanded ed.

5.3.5 Place of Publication and Publisher

1. For Thai-language references, include the name of the province as the place of publication. For Bangkok, use the term Bangkok.

2. If the publisher has offices in multiple cities and the names of those cities appear in the document, use the first city mentioned as the place of publication.

3. List the publisher's name without including terms like Publisher, Company Limited, "Publisher," "Co—Inc.," or "Co..Ltd."

4. If the document does not mention the publisher's name, list the name of the printing house with the term Printing House.

5. For publishers associated with universities or organizations, provide the full name, such as Thammasat University Press.

6. If the printer is a government or private organization, use the organization's name as the publisher.

7. If the name of the person responsible for printing is not stated, use 'n.p.' (no place of publishing) in English.

5.3.6 Year of Publication

1. Include the year of publication in numeric format, as it appears in the document.
2. If the publication year is not mentioned, use 'n.d.' (no date) in English.

5.3.7 Example of Bibliography Format

The specifications for referencing in the bibliography format are as follows:

1. Variables in different listings, such as author, day, month, year, and publisher, should use the regular Courier PC format.
2. Variables that need to be printed in bold should use the Courier Upright Bold format.
3. Terms that need to be printed in italics, such as 'ผู้แปล' (translator) or 'โดย' (by), should use the regular Arial Unicode MS format.
4. Replace single-letter spacing with the '/' symbol.
5. In cases where a single line is not sufficient, start the second line with the eighth character, leaving a space of 7 characters.
6. Other symbols such as '.', ',', " ", :, -, [ ], should be printed as specified.

5.3.7.1 Book

Format:

Author.//Publication Year.//Title.//Volume (if applicable).//Edition (if applicable).//City of Publication.//Publisher.

Examples:

Single Author:

Sumner, M. 1990. Computer: Concept and uses. 2nd ed. New York: McGraw-Hill.

Two Authors:

Gilbert, A. and Gnglir, J. 1982.**Cities Poverty and Development Urbanization in the Third World.** London: Oxford University Press.

Multiple Authors:

Jackson, M.H. Stewart, D. and Steven, G. 1991.**Environmental Health Reference Book.** Oxford: Butterworth Heineman.

Authors with Pseudonyms:

น.ม.ส. (นามแฝง). 2496. นิทานของ น.ม.ส. พิมพ์ครั้งที่ 3. พระนคร: คลังวิทยา.

5.3.7.2 Translated Books

Format

Author.//Year BE. Publication Year.//**Book Title.**//Volume (if any).//Translated by Translator.//Edition (if any).//City of Publication://Publisher.

Or

Author.//Year BE. Publication Year.//Book Title.//Volume (if any).//Translated from**Original Title**.//by Translator.//Edition (if any).//City of Publication://Publisher.

Example

Davenport, J.H. 1993.**Computer Algebra.** Translated by Davenport, A and Davenport, J.H. 2nd ed. Great Britain : Academic Press.

Shafarevich, I.R. 1994. Basic Algebraic Geometry 1. translated from **Osnovy Algebraicheskoj Geometrii, tom 1.** by Reid, L. 2nd ed. New York : Springer.

5.3.7.3 Translated Books with Unspecified Original Author in the Original Language

Format

Translator Name,/Translator.//Year BE. Publication Year.//Book Title.//Volume (if any).//Edition (if any).//City of Publication://Publisher.

Example

Duang Bunnak, Translator. 2511. Economics. 2nd ed. Bangkok: Thammasat University.

Apichat Thaowto and Serm Sakkulkul, Translators. 2526. Basic Knowledge of Rice Cultivation. Bangkok: Kasetsart University.

5.3.7.4 Books Authored by Organizations

Format

Organization Name.//Year BE. Publication Year.//Book Title.//Volume (if any).//Edition (if any).//City of Publication://Publisher.

Example

The Bank of Thailand. 1992. 50 Years of the Bank of Thailand 1942-1992. Bangkok : Amarin Printing Group.

5.3.7.5 Editors or Compilers

Format

Editor Name,/Editor (Compiler).//Year BE. Publication Year.//Book Title.//Volume (if any).//Edition (if any).//City of Publication://Publisher.

Example

Friedman, E.G., editor. 1995. Clock Distribution Networks in VLSI Circuits and Systems. New York : IEEE Press.

5.3.7.6 Conference Proceeding or Articles from Books

Format

Author of the Article.//Year BE. Publication Year.// ”Article Title (Conference Report).”//Page/Page Range.//In//Editor (Compiler).//Book Title.//Volume (if any).//Edition (if any).//City of Publication://Publisher.

Example

Niwa, E. 1992. ”Chemistry of surimi gellation.” 115-118. in Lainier, T.C. and Lee, C.M. Surimi Technology. New York : Marcel Dekker

Goemans, M.X. and Williamson, D.P. 1995. ”The Primal-dual Method for Approximation Algorithms and its Application to Network design Problems.” 69-87. in Hochbaum, D. Approximation Algorithms for NP-hard Problems. Boston : PWS.

5.3.7.7 Journal Articles

Format

Author.//Publication Year.// ”Article Title.”//Journal Name.//Volume(Issue)/:/Page Range.

Bentley, M Lee. 1986. ”The Role of Backcountry Experience in Middle School Environmental Education,” Dissertation Abstracts Internation. 46(10) : 2900-A-2901-A.

Choomchuay, S. 1994. ”On the Implementation of Finite Field Operation.” Ladkrabang Engineering Journal. 11(1) : 7-16.

Hitchcock, R.B. et. al. 1982. ”Timing Analysis of Computer Hardware.” IBM J. Res. Develop. 26(1) : 100-105.

Klein, P. et. al. 1995. ”An approximate max-flow min-cut relation for undirected multi-Commodity flow, with applications.” Combinatorica. 15(2) : 187-202.

5.3.7.8 Thesis and Dissertation

Format

Author.//Year BE. Publication Year.// ”Thesis Title.”//Degree Name (Full).

Example

Choomchuay, S. 1993. ”Algorithm and Architecture for Reed-Solomon Decoding.” Ph.D.Thesis of University of London and the Diploma of Membership of the Imperial College.

Burns, B. 1991. ”Performance Analysis and Optimization of Asynchronous Circuits.” Ph.D. Dissertation, California Institute of Technology.

5.3.7.9 Reviews

Format

Reviewer Name.//Review of//Title of the Reviewed Work.//by//Author Name.//In//Book Title (where the review appears).//Edition (if any).//Page Range.

Example

Chamnan Nakprasom. Review of the book Lai Suea Siam by S. Siwarak. Social Perspectives Journal, Volume 5(1), Issue 1 (June-August 2510): 139-141.

5.3.7.10 Articles in Encyclopedias or Dictionaries

Format

Author.//"Article Title."//Encyclopedia or Dictionary Name.//Volume (Publication Year)/://Page Range.

Example

Prapat Trinanong. 2513. "Chor Fa." Thai Encyclopedia by the Royal Institute, Volume 10: 6076-6080.

5.3.7.11 Newspaper Articles

Format

Author.//Publication Year, Date, Month.//"Article Title."//Author of the Article.//Newspaper Name.//Page/Page Range.

Example

Goleman, D. 1985, 21 May. "New Focus on Multiple Personality." New York Times. Pages c1-c6.

5.3.7.12 News Monitoring and Facts Checking in Newspapers Over a Period

Format

Newspaper Name.//Date, Month//Year of Publication - Date, Month//Year of Publication.

Example

Thai Rath. January 4, 2538 - February 5, 2538.

5.3.7.13 Newsletters, Photocopies, and Unpublished Documents

Format

Author.//Year BE. Publication Year.//"Document Title."//Edition (if any).//City of Publication://Publisher.//Document Copy.

Example

Sakallah, K.A. et. al. 1989. "Analysis and Design of Latch-Controlled Synchronous Digital Circuits." Ann Arbor, Mi, : Dept. of EECS University of Michigan Technical Report CSE-TR-31-89.

5.3.7.14 Government Publications and Other Official Documents

Format

Organization Name.//Publication Year.//Publication Title.//Edition (if any).//City of Publication://Publisher.

Example

Office of the Secretary-General of the House of Representatives. 2522. Reports of the 8th-9th Representative Assembly Regular Session Volume 3.Bangkok:Printing Office of the Cabinet.

5.3.7.15 Interviews

Format

Interviewee Name,//Interviewer.//Date, Month Year of the Interview.//Interviewer Name//Interview Title.//Interview Location.

Example

Manas Sangwarasilp Interviewed, April 18, 2541. Somsak Chumchuay, Interviewer. Interview on the Improvement Guidelines for Graduate Programs of the Institution. Graduate School, King Mongkut's Institute of Technology Ladkrabang.

5.3.7.16 Audiovisual Materials

1) Slides, Filmstrips, Videos, Tape Recordings, Compact Discs

Format

Creator.//Year BE. Production Year.//Title.//[Type of Medium].//Place of Production://Producer.

Example

Charoen Prokpun Ltd. 2525. Chicken Farming. [Video]. Bangkok: Charoen Prokpun Ltd.

Alan W.B. Compiler. 1984. Using the Oscilloscope. [Slide]. London : The Slide Centre.

RCA Records. 1984. Test Compact Disc ; Extensive Test for Audio Equipment. [Compact Disc]. Germany : RCA Ltd.

2) Maps

Format

Name [Map Type].//Publication Year.//Publication Title.//Publisher.

Example

Scotland [Photographic map]. Washington (DC) : National Geographic Society (US.)

5.3.7.17 Patents

Format

Patentee.//Invention Name.//Country of Patent.//Patent Number.//Date of Patent Grant.

Example

Kamder, A.D. Method and Apparatus for Constraining the Compaction of Components of a Circuit Layout. U.S patent no. 5636132, June 1997.

5.3.7.18 Electronic Media

Format

Author.//Publication Year.//Title.//[CD-ROM].//Place of Production://Producer.

or

Author.//Publication Year (or Retrieval Date).//Title.//[Online].//Available://Access Method and Data Location.

Examples

Daniel H., Anghileri. 1995. Secondary Mathematics and Special Education Needs. [CD-ROM]. New York: Casell.

Noam E.M. 1994 Telecommunication Policy Issue for the Next Century. [Online]. Available: Gopher://198.80.36.../global/telecom.txt.

Texas Instruments. 1998. Speech synthesis Processors. [Online]. Available: http://www.ti.com/sc/docs/msp/speech/dex.htm.

5.3.8 Example Formats for References Citation

5.3.8.1 Book

Format

Author.//Book Title.//Edition (if any).//City of Publication://Publisher.//Publication Year BE.

or

Editor Name./Editor (Compiler).//Book Title.//Edition (if any).//City of Publication://Publisher.//Publication Year BE.

Example

[1] Richard E. Blahut. Theory and Practice of Error Control Codes. Reading : Addition-Wesley publishing Company, Inc.1984.

[2] Judith S.Van ALSTYNE. Professional and Technical Writing Strategies. International Edition. New JERSEY : Prentice-Hall, Inc. 1986.

[3] Jose E. Franca, Yannis Tsividis, Editors. Design of Analog-Digital VLSI Circuits for Telecommunication and Signal Processing. 2nd ED. New Jersey : Prentice-Hall, Inc.1994

[4] Microsoft. Hardware Design Guide for Window 95 ; A Practical Guide for Developing Plug and Play PCs and Peripherals. Washington D.C. : Microsoft Press. 1994.

[5] Microelectronics Group, Lucent Technology. Field-Programmable Gate Arrays [data Book]. LUCENT Technology. October 1996. Pp.2-36-2-40.

5.3.8.2 Articles from Journals, Books, or Conference Proceedings

Format

Author.//"Article Title."//Journal Name.//Volume,/Issue Number.//Month Year of Printing.//Page/Page Range.

Example

[1] Choomchuay S. "On the Implementation of Finite Field Operations" Ladkrabang Engineering Journal, vol.ll,no, l, June 1984. pp.7-17.

[2] Shao M., Reed S. "On the VLSI Design of a Pipeline Reed-Solomon Decoder Using Systolic Arrays." IEEE Trans. On Comput., vol. C-37, no. 10, Oct. 1988. pp. 1273-1280

[3] Sukiyama Y., Kasahara M., Hirasawa S. and Namekawa T. "A Method for Solving key Equation for Decoding Coppa Codes.", Inf. And Cont., vol.27, 1975. Pp. 87-99

[4] Choomchuay S., Arambepola B. "An Algorithm and a VLSI Architecture for Reed – Solomon Decoding" Proc. IEEE-ISCAS, San Diego, USA, May, 1992. Pp.2120-2123

5.3.8.3 Thesis and Dissertation

Format

Author.// ”Thesis Title.”// ”Full Degree Name.”//Year of Publication BE.

Example

[1] Choomchuay S. ”Algorithm and Architecture for Reed-Solomon Decoding.” Ph.D. Thesis of University of London and the Diploma of Membership of the Imperial College. 1993

5.3.8.4 Patents

Format

Patent Holder.//Invention Title.//Country of Patent.//Patent Number.//Date of Patent Registration (Day, Month, Year).

Example

[1] Kandar A.D. ”Method and Apparatus for Constraining the Compaction of Components of a Circuit Layout.” U.S patent no. 5636132, June 1997

5.3.8.5 Visual Materials and Other Media

Format

Producer.// ”Title.” //[Type of Media].//Place of Production.//Producer.//Year of Production BE.

Example

[1] RCA Records. ”Test Compact Disc ; Extensive Test Audio Equipment.” [Compact Disc]. Germany : RCA Ltd. 1984.

[2] Alan W.B. Compiler. ”Using the Oscilloscope.” [Slide]. London : The Slide Center. 1984.

5.3.8.6 Electronic Media

Format

Author.// ”Title.” //[CD-ROM].// Place of Production.//Producer.//Year of Release BE.

Or

Author.// ”Title.” //[Online].//Available at://://Access Method and Source.//Year of Publication BE (or retrieval).

Example

[1] Daniel H., Anghileri. ”Secondary Mathematics and Special Education Needs.” [CD-ROM]. New York : Casell.1995

[2] Noam E.M. ”Telecommunication Policy Issue for the Next Century.” [Online]. Available : gopher://198.80.36.../global/telcom.txt. 1994.

[3] Texas Instruments. ”Speech Synthesis Processors.” [Online]. Available : http://www.ti.com/sc/does/msp/speech/index.htm. 1998.

[1]

Bibliography

[1] Xuechen Zhang, Zhengmin Wu, Chengmao Cao, Kun Luo, Kuan Qin, Yangyang Huang, and Jie Cao. Design and operation of a deep-learning-based fresh tea-leaf sorting robot. *Computers and Electronics in Agriculture*, 206:107664, 2023.

Appendix A

GUIDELINES AND PROCEDURES FOR THESIS WRITING

Thesis writing is considered a crucial part for graduate students after completing coursework at a certain level. Students must plan their thesis writing process until the comprehensive thesis examination is completed.

Procedures for Requesting Approval of Thesis Topics and Scheduling Thesis Examinations:

1. Graduate students have the right to propose thesis topics as soon as they become enrolled in the graduate program.

2. Students must submit a request for approval of the thesis topic to the graduate school of their respective faculty.

3. The approval of the thesis topic and the thesis outline is within the authority of the dean, endorsed by the academic affairs committee.

4. Students must receive approval of the thesis topic and the outline (if any) before the final thesis examination.

5. Any changes before the thesis examination must be handled as follows:

5.1 **Change of thesis topic or outline:** Students must submit a request to the dean, with approval from the thesis advisor and endorsement from the academic affairs committee.

5.2 **Change of thesis advisor:** Students must submit a request to the dean of the respective academic unit, and the dean has the authority to approve, with endorsement from the academic affairs committee.

5.3 In cases where the thesis advisor loses their position within the academic unit, the student must find a new advisor, and the work specified by the owner of the work, including the student's name, the name of the thesis advisor, the name of the academic unit, and the name of the institution, should be used to support the thesis examination application.

6. The thesis examination is the final stage, and students must comply with the conditions and requirements of each program in their respective academic units. Students can only take the thesis examination when:

6.1 The thesis advisor believes that the student is ready for the examination. The student must notify the examination committee and submit a draft of the thesis to the specified number of graduate students, faculty, and the examination committee no less than two weeks in advance.

6.2 For master's degree examinations, the branch chairman and the course coordinator must conduct the thesis examination committee, endorsed by the academic affairs committee. The committee should consist of five members, including the thesis advisor, one graduate faculty member proposed by the thesis advisor, and three members, including an outside faculty member who is an expert, proposed by the academic affairs committee. The committee must appoint the external committee member as the committee chairman.

6.3 For doctoral degree examinations, the dean should appoint the thesis examination committee, endorsed by the academic affairs committee. The committee should consist of five members, including the thesis advisor, one faculty member proposed by the thesis advisor, and three members, including an outside faculty member who is an expert, proposed by the academic affairs committee. The committee must appoint the external

committee member as the committee chairman.

AUTHOR BIOGRAPHY

Name Mr. First Name Last Name

Date of Birth January 1, 1990, in Bangkok

John Doe was born in Bangkok, Thailand, on January 1, 1995. He received his Bachelor of Engineering degree in Robotics and AI Engineering from KMITL in 2017. He is currently pursuing a Master of Engineering degree in Robotics and AI Engineering at KMITL. His research interests include machine learning, computer vision, and robotics.